Memory Slice (.msl)

Binary Format Specification

Version: 1.1.1
Status: Working Draft
Editor(s): Daniel Baier
This version: https://github.com/MemorySlice/memslice-spec/releases/tag/v1.1.1
Latest version: https://github.com/MemorySlice/memslice-spec/

Abstract

Memory Slice (`.msl`) is a self-describing, block-based binary format for capturing the forensic state of a single operating system process. It records both the virtual address space (with per-page acquisition status) and transient OS-queryable metadata that exists only while the process is alive. This document specifies the binary layout: file header, block architecture, integrity chain, capture-time payloads, extension segments for post-acquisition enrichment, cross-referencing, capability bitmap, raw dump import mechanism, investigation mode with system-wide context capture, and full-container AEAD encryption with post-quantum key encapsulation support.

This specification accompanies the research paper *Memory Slice: A Process-Centric Dump Format for Differential Memory Analysis and Volatility 3 Plugin Synthesis.*

Contents

1 Scope and Conventions **3**
1.1 Scope . . . 3
1.2 Normative Language . . . 3
1.3 Encoding Conventions . . . 3
1.4 Byte Order Domains . . . 3
1.5 UUID Requirements . . . 4
1.6 Terminology . . . 4

2 Format Overview **6**
2.1 Acquisition Modes . . . 6
2.2 Design Principles . . . 7
2.3 Scope of the Integrity Guarantee . . . 7

3 File Header **9**
3.1 Base Header (64 bytes) . . . 9
3.2 Encryption Extension Header (`0x40`–`0x7F`) . . . 10
3.3 Byte Order . . . 11
3.4 Version Compatibility . . . 12

4 Block Architecture **13**
4.1 Common Block Header (80 bytes) . . . 13
4.2 Block Flags . . . 14
4.2.1 Compression Semantics . . . 14
4.3 Block Type Registry . . . 14
4.4 Integrity Chain (`PrevHash`) . . . 15
4.5 End-of-Capture Block (`0x0FFF`) . . . 16
4.6 Extension Segments and End-of-Extension Block (`0x0FFE`) . . . 17

5 Capture-Time Payloads **20**
5.1 Memory Region (`0x0001`) . . . 20
5.2 Module Entry (`0x0002`) . . . 20
5.3 Module List Index (`0x0010`) . . . 21
5.4 Process Identity (`0x0040`) . . . 22
5.5 Related Dump (`0x0041`) . . . 22
5.6 Key Hint (`0x0020`) . . . 23
5.7 Thread Context (`0x0011`) . . . 24
5.8 VAS Map (`0x1001`) . . . 27
5.9 Physical Memory Map (`0x0059`) . . . 28
5.10 Module Build-ID Manifest (`0x005A`) . . . 29
5.11 Region Omission Manifest (`0x005B`) . . . 30

6 Investigation Mode **33**
6.1 Block Ordering . . . 33
6.2 System Context Block (`0x0050`) . . . 33
6.3 Process Table (`0x0051`) . . . 35
6.4 Connection Table (`0x0052`) . . . 36
6.5 Handle Table (`0x0053`) . . . 36
6.6 Connectivity Table (`0x0054`) . . . 37

6.7 Redaction and Data Minimization 38

7 Page State Map **39**

8 Block Cross-Referencing **39**

9 Capability Bitmap **40**

10 Encryption **41**
10.1 Scope 41
10.2 Encrypted File Layout 41
10.3 Cipher Suites 42
10.4 Key Encapsulation 42
10.5 Key Derivation 43
10.6 Interaction with the Integrity Chain 43
10.7 Streaming Encryption 43
10.8 Sealed Extension Segments 44

11 Raw Dump Import **46**

12 Parsing Walkthrough **46**

13 Worked Example **48**

14 Conformance **49**
14.1 Producer 49
14.2 Consumer 50

15 Revision History **51**

1 Scope and Conventions

1.1 Scope

This document defines the binary wire format of Memory Slice (`.msl`) files. It does *not* specify acquisition procedures or analysis algorithms.

1.2 Normative Language

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, "RECOMMENDED", **MAY**, and **OPTIONAL** are per RFC 2119 [1].

1.3 Encoding Conventions

- All multi-byte structural integers are **little-endian**. The file header's `Endianness` byte at offset `0x08` **MUST** be `0x01` in version 1.1. The field is retained for forward compatibility; future versions **MAY** define `0x02` (big-endian). Consumers encountering a value other than `0x01` **MUST** reject the file.
- Strings are UTF-8, null-terminated, padded to **8-byte** boundaries with zero bytes. The pad function is defined as $\text{pad8}(n) = \lceil n/8 \rceil \times 8$, where n is the byte length of the UTF-8 encoded string **including** the mandatory terminating NUL (`0x00`). All padding bytes **MUST** be `0x00`. Textual string fields **MUST NOT** contain `0x00` bytes before the terminating NUL.
- Where a string field is accompanied by an explicit length field (e.g., `PathLen`, `ExePathLen`), the **length field is authoritative**: it specifies the byte count including the terminating NUL. The on-disk storage occupies pad8(Len) bytes. Consumers **MUST** use the length field to determine string extent and **MUST NOT** scan beyond `Len` bytes. The NUL byte at position Len − 1 serves as a consistency check.
- Producers **MUST** emit well-formed UTF-8 per RFC 3629 [3]. Consumers encountering invalid UTF-8 sequences **SHOULD** preserve the raw bytes and **MAY** flag the field as malformed, but **MUST NOT** reject the entire block. This ensures that forensic evidence is never silently discarded due to encoding errors in captured metadata.
- Variable-length fields (`PageStateMap`, `NativeBlob`) are padded to 8-byte boundaries.
- UUIDs **MUST** be version 4 (random) per RFC 9562 [2], stored as 16 raw bytes in RFC 9562 binary layout. See Section 1.5.
- Integrity hashes use the algorithm selected by `HashAlgo` (Section 4.4); all registered algorithms produce 32-byte output. The default is BLAKE3 [4].
- Timestamps: unsigned 64-bit nanoseconds since Unix epoch (1970-01-01T00:00:00Z).
- All `Reserved` fields **MUST** be zero (producers) and **MUST** be ignored (consumers).

1.4 Byte Order Domains

Three byte-order regimes coexist within an MSL file. Implementers **MUST** observe the boundaries precisely:

1. **Structural integers.** All multi-byte integers in the file header, block headers, and payload fixed fields are little-endian (controlled by `Endianness`=`0x01`).

2. **UUIDs.** All UUID fields are stored in RFC 9562 binary layout (big-endian mixed-endian per the RFC) and are *never* subject to byte-swapping, regardless of the file's `Endianness` setting.
3. **Captured memory content.** Bytes within `PageData` are stored verbatim in the target architecture's native byte order; the format does not reorder them.

1.5 UUID Requirements

All UUID fields (`DumpUUID`, `BlockUUID`, `ParentUUID`, payload UUIDs) **MUST** be version 4 (random) per RFC 9562. The 4-bit version field (bits 48–51) **MUST** be `0100`; the 2-bit variant (bits 64–65) **MUST** be `10`. UUIDs are stored as 16-byte sequences in RFC 9562 binary layout and are never subject to byte-swapping.

Implementation guidance: UUID generation

A thread-local fast PRNG such as `xoshiro256++` or `PCG64`, seeded once from OS entropy, provides sufficient uniqueness ($\sim 2^{-61}$ collision probability per pair) while avoiding CSRNG overhead per block. Note that this is a *performance optimization*, not a conformance requirement; implementations **MAY** use a CSRNG if preferred.

1.6 Terminology

Producer
Software that creates or appends blocks.

Consumer
Software that reads an MSL file.

Acquirer
A producer capturing live process state (types `0x0000`–`0x0FFF`).

Importer
A producer converting a raw dump into MSL (Section 11).

Enrichment tool
A producer that appends extension segments (Section 4.6) to an existing file.

Block
Fixed header + variable payload.

Payload
Block-type-specific data after the header.

Preamble
Block-level fields at the start of a payload that precede repeated entry structures (e.g., `EntryCount`).

Capture segment
Block 0 through the End-of-Capture block: the evidence as written by the acquirer or importer (Section 4.5).

Extension segment
One or more blocks appended after the capture segment and terminated by an End-of-Extension block (Section 4.6).

Investigation Mode
Acquisition for forensic investigations with system-wide context (Section 6).

Analysis Mode
Default mode for research and tool development. No system-wide context required.

The following sections define the format itself, starting with the high-level file structure.

2 Format Overview

An MSL file is a linear byte stream: a **file header** followed by **typed, length-prefixed blocks**. No inter-block gaps. `HeaderSize` tells the consumer where blocks begin. Figure 1 shows the high-level structure.

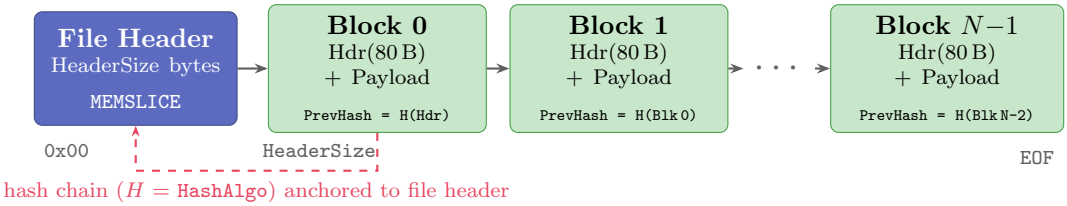


Figure 1: MSL file structure: file header followed by a hash-chained capture segment (terminated by the End-of-Capture block) and optional extension segments (Section 4.6).

Block 0's `PrevHash` = H(File Header), where H is the hash selected by `HashAlgo` (BLAKE3 by default; Section 4.4). Each subsequent block hashes its predecessor. The capture segment ends with the End-of-Capture block (Section 4.5) and **MAY** be followed by extension segments (Section 4.6). Consumers detect end-of-file when fewer than 80 bytes remain or the next 4 bytes do not match the block magic (`0x4D534C43`). Every non-zero block count—in the file header or in the EoC block—**MUST** equal the number of capture-segment blocks actually parsed (Section 3.1, *BlockCount semantics*).

2.1 Acquisition Modes

The file header's `Flags` field (offset `0x0C`) determines the acquisition mode and whether encryption is active:

Analysis Mode (`Investigation=0`)
: The default. For research and developing new forensic approaches. Header is 64 bytes. No system-wide context required. Figure 2.

Investigation Mode (`Investigation=1`)
: For forensic investigations. Block 2 **MUST** be a System Context block (`0x0050`). Section 6. Figure 3.

Either mode **MAY** set the `Encrypted` flag (bit 2), activating full-container Authenticated Encryption with Associated Data (AEAD) as defined in Section 10. When set, `HeaderSize` is 128 and the entire block stream is encrypted. Table 1 summarizes the combinations.

Table 1: Acquisition mode and encryption flag combinations.

Investigation	Encrypted	HeaderSize	Block 2
0	0	64	Memory Region or other capture-time block
0	1	128	Memory Region or other (encrypted)
1	0	64	System Context (`0x0050`, **REQUIRED**)
1	1	128	System Context (`0x0050`, **REQUIRED**, encrypted)

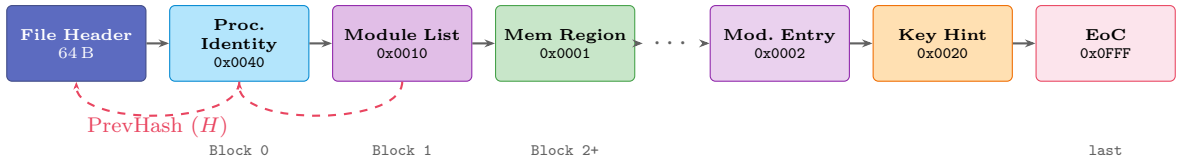


Figure 2: Analysis mode (`Investigation`=0, `Encrypted`=0). Thread Context blocks (`0x0011`), when present, precede the first Memory Region (Section 5.7).

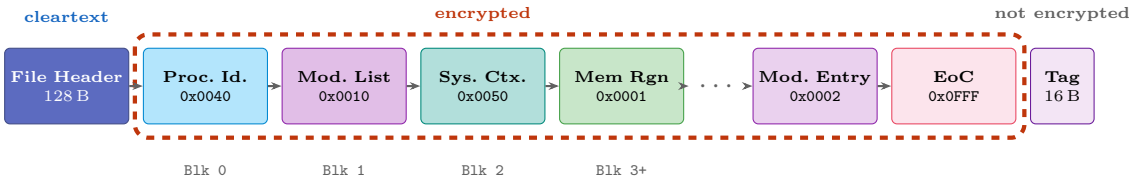


Figure 3: Investigation mode with encryption (`Investigation`=1, `Encrypted`=1). The 16-byte authentication tag is an *output* of the AEAD, not itself encrypted.

2.2 Design Principles

1. **Integrity Verification.** Append-only policy with a hash chain (BLAKE3 by default).
2. **Epistemic Honesty.** Four-state pages: Captured vs. Failed vs. Unmapped vs. NotResident.
3. **Self-Describing.** Version, header size, OS, arch, PID, capabilities.
4. **Dual-Layer OS Abstraction.** Normalized fields + OS-native opaque blob.
5. **Dual Acquisition Modes.** Analysis mode for research; Investigation mode for forensic investigations.
6. **Confidentiality.** Full-container AEAD encryption with post-quantum key encapsulation.

Capture-time blocks are optimized for streaming, append-only acquisition. The Structural namespace (`0x1000`–`0x1FFF`) holds derived views of the capture—virtual address maps, seek tables, pointer graphs—that enable efficient random access. Such blocks are written either at capture time (before the End-of-Capture block) or afterwards in extension segments (Section 4.6), which leave the capture segment byte-for-byte untouched.

2.3 Scope of the Integrity Guarantee

The hash chain provides *tamper detection*, not *tamper resistance*. It protects against **accidental corruption** and detects **naive modification**, including block reordering, insertion, and deletion—threats that a per-block checksum alone cannot detect, since the chain cryptographically binds each block to its predecessor and ultimately to the file header. For tamper evidence admissible in legal contexts, producers **SHOULD** complement the hash chain with external mechanisms (digital signature over `FileHash`, RFC 3161 timestamp, write-once media). When encryption is active, the AEAD tag provides stronger integrity. The hash chain alone **MUST NOT** be represented as providing forensic-grade tamper evidence.

Table 2: Integrity mechanisms: guarantees and limitations.

Mechanism	**Protects Against**	**Limitations**	**Key?**
`PrevHash` chain	Accidental corruption, naive modification	Attacker can recompute chain after modification	No
EoC `FileHash`	Truncation, missing blocks	Recomputable by attacker; only verifiable after decryption if encrypted	No
Extension `SegmentHash` + `BlockCount`	Truncated or unterminated extension segments	Recomputable by attacker; covers only the segment itself	No
AEAD tag	Any tampering of encrypted block stream	Header remains cleartext (metadata leakage); requires key compromise to forge	Yes
External hash	Complete file substitution, transport corruption	Only as trustworthy as the storage of the hash itself	No

3 File Header

3.1 Base Header (64 bytes)

Table 3: File header (64 bytes minimum; 128 when `Encrypted`).

Offset	Size	Field	Abbr	Description
0x00	8	Magic	—	0x4D454D534C494345 ("MEMSLICE").
0x08	1	Endianness	E	v1.1: **MUST** be 0x01 (LE). 0x02 (BE) reserved for future versions. Other values **MUST** cause rejection.
0x09	1	HeaderSize	HS	0x40 (64) for unencrypted; 0x80 (128) when `Encrypted`. First block (or Key Encapsulation Mechanism ciphertext) starts here.
0x0A	2	Version	Ver	uint16. Major in high byte, minor in low: v1.1 = 0x0101.
0x0C	4	Flags	—	See Table 4.
0x10	8	CapBitmap	CapBm	Capability bitmap (Section 9).
0x18	16	DumpUUID	—	UUIDv4 uniquely identifying this dump.
0x28	8	Timestamp	Ts	Acquisition start (UTC, ns since epoch).
0x30	2	OSType	OS	Target OS (Table 6a).
0x32	2	ArchType	Ar	Target CPU (Table 6b).
0x34	4	PID	—	Process ID at acquisition.
0x38	1	ClockSource	CS	0x00=Unknown, 0x01=CLOCK_REALTIME, 0x02=CLOCK_MONOTONIC_RAW, 0x03=QPC, 0x04=mach_absolute_time. 0xFF=Other.
0x39	4	BlockCount	BCt	Number of blocks in the capture segment (incl. EoC). 0x00000000 = unknown. Never patched after writing; see *BlockCount semantics* below and Section 4.5.
0x3D	1	HashAlgo	HA	Integrity hash algorithm selector (Table 12). 0x00=BLAKE3 (default).
0x3E	2	Reserved	Rsv	**MUST** be zero.

Size: 8+1+1+2+4+8+16+8+2+2+4+1+4+1+2 = 64.

BlockCount semantics

`BlockCount` counts the blocks of the capture segment, including the End-of-Capture block; blocks in extension segments (Section 4.6) are never counted. A producer that does not know the count when it writes the header (streaming acquisition) **MUST** write 0x00000000 and

MUST NOT patch the field afterwards: the header is covered by Block 0's `PrevHash` and by the EoC `FileHash`, so a later modification (except the one-time `CipherLen` patch, Section 10.7) would invalidate both. Such a producer **SHOULD** record the exact count in the EoC block's `BlockCount` field instead (Table 13). Encrypted containers **MAY** leave the header field zero and rely on the (encrypted) EoC count. Every non-zero count—header or EoC—**MUST** equal the number of capture-segment blocks actually parsed, including the EoC block; otherwise the file **MUST** be considered truncated or corrupt (consumer rule 13, Section 14).

Table 4: File header flags (`Flags`, 32 bits at offset `0x0C`).

Bit	Name	Description
0	`Imported`	File created by importing a raw dump (Section 11).
1	`Investigation`	Forensic investigation. System Context block (`0x0050`) **MUST** be Block 2 (Section 6).
2	`Encrypted`	Full-container AEAD encryption. `HeaderSize` **MUST** be 128 (Section 10).
3	`Redacted`	One or more fields have been intentionally stripped or pseudonymized (Section 6.7).
4–31	Reserved	Zero.

When `Investigation` is set and `Encrypted` is not, confidentiality **SHOULD** be ensured by external means (e.g., full-disk encryption, physically secured storage). Unencrypted Investigation mode is appropriate when the goal is rapid triage or when organizational policy provides confidentiality at the storage layer.

3.2 Encryption Extension Header (`0x40`–`0x7F`)

When `Encrypted` is set, `HeaderSize` **MUST** be 128. Bytes `0x00`–`0x3F` retain the base layout; bytes `0x40`–`0x7F` carry encryption parameters (Table 5).

Table 5: Encryption extension header (64 bytes at `0x40`–`0x7F`).

Off	Size	Field	Abbr	Description
0x40	1	EncAlgo	EA	AEAD cipher suite (Table 38).
0x41	1	KDFType	KD	0x00=None, 0x01=Argon2id.
0x42	1	KeyEncap	KE	Key encapsulation (Table 39). 0x00=None.
0x43	1	Reserved	R	Zero.
0x44	4	KDFTime	KTm	Argon2id time cost. 0 if N/A.
0x48	4	KDFMemory	KMm	Argon2id memory (KiB). 0 if N/A.
0x4C	1	KDFLanes	L	Argon2id parallelism. 0 if N/A.
0x4D	1	Reserved2	R	Zero.
0x4E	2	KEMCtLen	KCL	KEM ciphertext length. 0 when KeyEncap=0x00.
0x50	24	Nonce	—	AEAD nonce. AES-256-GCM: 12B used, rest zero. XChaCha20-Poly1305: all 24B.
0x68	16	KDFSalt	—	KDF salt. Zeros if N/A.
0x78	8	CipherLen	CL	Byte length of the primary AEAD ciphertext (Block 0 through EoC plus in-stream extension segments). 0 = not recorded. Excluded from the AAD (Section 10.2).

Size: 1+1+1+1+4+4+1+1+2+24+16+8 = 64. Total header: 64+64 = 128.
The extension carries all parameters needed to decrypt the block stream: the cipher suite, KDF configuration (for passphrase-derived keys), Key Encapsulation Mechanism (KEM) ciphertext length, the AEAD nonce, and the KDF salt. Figure 4 shows the combined 128-byte header layout with base and extension regions.

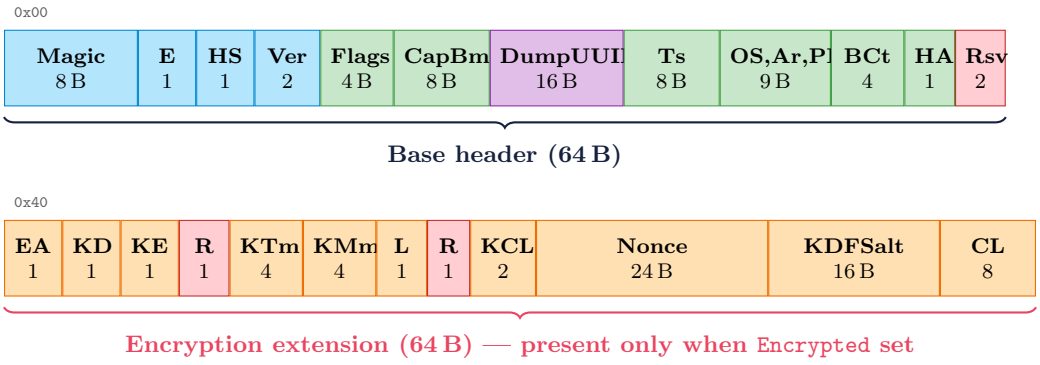


Figure 4: Extended file header (128 bytes). Abbreviations per Tables 3 and 5.

3.3 Byte Order

All multi-byte structural integers are little-endian (`Endianness=0x01`). Memory content within `PageData` is stored in the target architecture's native order. See Section 1.4 for the complete

byte-order domain summary.

3.4 Version Compatibility

Unknown major version: **SHOULD** reject. Unknown minor of a known major: **MUST** parse; minor revisions only add block types, reserved-field semantics, and optional structures, all of which a consumer of an earlier minor skips by the usual rules (unknown block types via `BlockLength`, reserved fields ignored). Files that use any feature introduced in version 1.1 (extension segments, EoC `BlockCount`, `HashAlgo`, Thread Context, VAS Map `NameSource`, blocks `0x0054` and `0x0059`–`0x005B`, page state NotResident) **SHOULD** set `Version` to `0x0101`; files written as `0x0100` remain valid. Patch revisions (e.g., 1.1.1) do not change `Version`, which stays `0x0101` (there is no patch byte); they correct errata and define semantics for reserved fields whose zero value preserves the prior meaning. A 1.1.0 consumer computes the AAD over the unzeroed header and therefore cannot decrypt files with `CipherLen`≠0 (Section 10.2); files with `CipherLen`=0 are unaffected. A consumer whose tag verification fails on such a file **SHOULD** report that it may require 1.1.1 support rather than tampering (Section 10.2). Always use `HeaderSize` to locate Block 0.

Table 6: Target platform codes: (a) OS and (b) CPU.

(a) OS type codes (`OSType`)

Code	OS
`0x0000`	Windows
`0x0001`	Linux
`0x0002`	macOS
`0x0003`	Android
`0x0004`	iOS/iPadOS
`0x0005`	FreeBSD
`0x0006`	NetBSD
`0x0007`	OpenBSD
`0x0008`	QNX
`0x0009`	Fuchsia
`0x000A--0x00FF`	*Reserved*
`0x0100--0xFFFE`	*Vendor*
`0xFFFF`	Unknown

(b) Architecture codes (`ArchType`)

Code	Architecture
`0x0000`	x86 (IA-32)
`0x0001`	x86_64 (AMD64)
`0x0002`	ARM64 (AArch64)
`0x0003`	ARM32
`0x0004`	MIPS32
`0x0005`	MIPS64
`0x0006`	RISC-V RV32
`0x0007`	RISC-V RV64
`0x0008`	PPC32
`0x0009`	PPC64
`0x000A`	s390x
`0x000B`	LoongArch64
`0x000C--0x00FF`	*Reserved*
`0x0100--0xFFFE`	*Vendor*
`0xFFFF`	Unknown

4 Block Architecture

Every block consists of a **fixed 80-byte Common Block Header** followed by a **variable-length payload**. `BlockLength` specifies the total **on-disk** size of the block (header + payload, in compressed form if applicable). Consumers **MUST** verify `BlockLength` ≥ 80 and **MUST** reject blocks that violate this constraint. Unknown types are skipped via `BlockLength`.

4.1 Common Block Header (80 bytes)

Every block begins with the same 80-byte header, which provides type identification, length framing, UUID-based identity and parentage, and the integrity chain anchor. The fixed header size ensures that consumers can always parse block boundaries without understanding payload formats. Table 7 defines the fields; Figure 5 shows the layout.

Table 7: Block header (80 bytes). All fields ≥ 8 B are 8-byte aligned.

Off	Size	Field	Abbr	Description
0x00	4	Magic	—	0x4D534C43 ("MSLC").
0x04	2	BlockType	Type	Payload selector (Section 4.3).
0x06	2	Flags	—	Per-block flags (Table 8).
0x08	4	BlockLength	Len	Total on-disk size (hdr+payload). ≥ 80.
0x0C	2	PayloadVersion	PVer	Default: 0x0001.
0x0E	2	Reserved	R	Zero.
0x10	16	BlockUUID	BlkUUID	UUIDv4. 8-byte aligned.
0x20	16	ParentUUID	ParUUID	Parent or zeros. 8-byte aligned.
0x30	32	PrevHash	—	*H* (HashAlgo) of preceding element. Zeros when encrypted. 8-byte aligned.
0x50	var	Payload	—	BlockLength − 80 bytes.

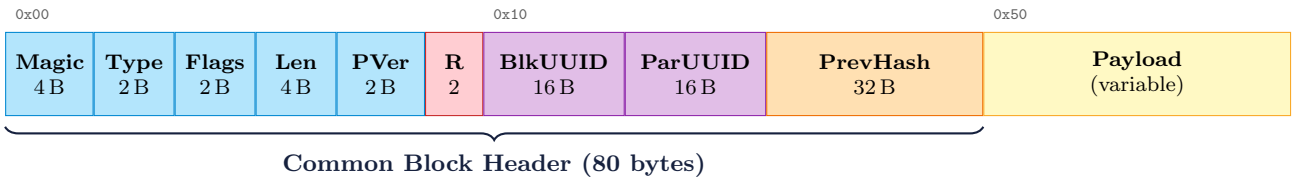


Figure 5: Common Block Header layout. Abbreviations per Table 7.

4.2 Block Flags

Table 8: Block flags (16-bit).

Bit(s)	Name	Description
0	`Compressed`	Algorithm in bits 1–2.
1–2	`CompAlgo`	00=none, 01=zstd, 10=lz4, 11=reserved.
3	`HasKeyHints`	Key Hint blocks reference this block.
4	`HasChildren`	Referenced via `ParentUUID`.
5	`Continuation`	Continuation fragment.
6–15	Reserved	Zero.

4.2.1 Compression Semantics

When the `Compressed` flag is set, only the payload portion (bytes after the 80-byte Common Block Header) is compressed. The on-disk compressed payload is structured as follows:

Table 9: Compressed payload framing.

Off	Size	Field	Description
+0x00	8	`UncompressedSize`	`uint64`. Exact size of the decompressed payload in bytes.
+0x08	var	`CompressedData`	Compressed byte stream (zstd or lz4).

`BlockLength` reflects the total on-disk size: 80+8+len(`CompressedData`). After decompression, the resulting `UncompressedSize` bytes are parsed according to the block type's payload definition. The block header (including `BlockType`, `Flags`, and `BlockLength`) is never compressed. The hash chain covers the on-disk (compressed) bytes.

4.3 Block Type Registry

Table 10: Block type namespaces.

Range	Namespace	Writer	Examples
0x0000–0x0FFF	Capture-Time	Acquirer/Importer in the capture segment; enrichment tools for the types Section 4.6 permits in extension segments	Regions, modules, keys, system context
0x1000–0x1FFF	Structural	Acquirer, importer, or analysis tool; capture segment or extension segment	VAS map, pointer graph
0x2000–0xFFFF	Reserved	—	Future versions

Table 11: Defined block types.

Type	Name	Description
0x0000	*Invalid*	**MUST NOT** appear.
0x0001	Memory Region	Per-page memory (Sec. 5.1).
0x0002	Module Entry	Module metadata (Sec. 5.2).
0x0010	Module List Index	Module manifest (Sec. 5.3). **MUST** be Block 1.
0x0011	Thread Context	Register state (Sec. 5.7).
0x0012	File Descriptor	Open handle.
0x0013	Network Connection	Socket attribution.
0x0014	Environment Block	Env vars.
0x0015	Security Token	Credentials.
0x0020	Key Hint	Crypto key annotation (Sec. 5.6).
0x0030	Import Provenance	Raw dump metadata (Sec. 11).
0x0040	Process Identity	Exe path, cmdline, PPID (Sec. 5.4).
0x0041	Related Dump	Cross-file reference (Sec. 5.5).
0x0050	System Context	Investigation metadata (Sec. 6.2). Block 2 when `Investigation` set.
0x0051	Process Table	System-wide process list (Sec. 6.3).
0x0052	Connection Table	System-wide sockets (Sec. 6.4).
0x0053	Handle Table	System-wide handles (Sec. 6.5).
0x0054	Connectivity Table	Kernel network state without socket endpoints (Sec. 6.6).
0x0059	Physical Memory Map	Physical address ranges of the host (Sec. 5.9).
0x005A	Module Build-ID Manifest	Build IDs and disk hashes merged into the module list (Sec. 5.10).
0x005B	Region Omission Manifest	Regions deliberately skipped by the acquirer (Sec. 5.11).
0x0FFE	End-of-Extension	Extension segment terminator (Sec. 4.6).
0x0FFF	End-of-Capture	Completeness marker (Sec. 4.5).
0x1001	VAS Map	Enumerated virtual address space (Sec. 5.8).
0x1003	Pointer Graph	Pointer relationships. Producer-defined payload; extension segments only (Sec. 4.6).

Block types listed without a section reference (0x0012–0x0015) are reserved evidence types whose payload formats will be defined in future versions of this specification. Producers **MUST NOT** emit them anywhere until their payloads are specified; they are also excluded from extension segments. Block types whose payload this document does not specify but which are not evidence—the Pointer Graph (0x1003) and other producer-defined Structural types—**MAY** be emitted only in extension segments (Section 4.6), where a consumer that does not recognize the payload skips the block. Consumers **MUST** skip unknown types via `BlockLength`.

4.4 Integrity Chain (`PrevHash`)

The integrity hash algorithm is selected by the `HashAlgo` field in the file header (Table 3). All integrity fields within a single file—`PrevHash`, EoC `FileHash`, End-of-Extension `SegmentHash`, and Module Entry `DiskHash`—**MUST** use the same algorithm; producers **MUST NOT** mix algorithms within a file. All registered algorithms produce a 32-byte output, so field layouts are independent of the selection.

Let $H(\cdot)$ denote the selected algorithm. Then:

- **Block 0:** `PrevHash` = H(File Header, all `HeaderSize` bytes).

- **Block i ($i \geq 1$):** `PrevHash` $= H(\text{Block}_{i-1})$. The chain continues across the End-of-Capture block into extension segments (Section 4.6).

Table 12: Integrity hash algorithm codes (`HashAlgo`).

Code	Algorithm	Output	Notes
`0x00`	BLAKE3 [4]	32 B	**Default.** High throughput via tree hashing and SIMD.
`0x01`	SHA-256 [5]	32 B	FIPS 180-4. Hardware-accelerated on Intel SHA-NI, AMD Zen, and ARMv8 Cryptographic Extension.
`0x02`	SHA-512/256 [5]	32 B	FIPS 180-4. Truncated SHA-512 with distinct initial values; not length-extension equivalent to SHA-512. Often faster than SHA-256 on 64-bit platforms lacking SHA-NI.
`0x03--0xFE`	*Reserved*	—	Reserved for future algorithms.
`0xFF`	Other	32 B	Non-standard; interpretation out of scope.

Default and rationale. `HashAlgo`=`0x00` (BLAKE3) is **RECOMMENDED** for general use and is the default for all implementations. Producers **SHOULD** select `0x01` (SHA-256) or `0x02` (SHA-512/256) only when FIPS 140 compliance, ecosystem interoperability, or organizational policy requires a NIST-standardized hash; or when platform measurements indicate a NIST hash outperforms BLAKE3—for example, aarch64 systems with the ARMv8 Cryptographic Extension, where SHA-256 hardware acceleration can outperform software BLAKE3.
Scope of selection. The `HashAlgo` selection applies only to integrity hashing: `PrevHash`, `FileHash`, `SegmentHash`, and `DiskHash`. The key derivation function defined in Section 10.4 always uses HKDF-BLAKE3 regardless of `HashAlgo`; integrity hashing and key derivation are independent design choices and are not configured together.
Interpretation of references. Where this specification refers to "BLAKE3" in integrity contexts—`PrevHash` (this section), `FileHash` (Section 4.5), `SegmentHash` (Section 4.6), and `DiskHash` (Section 5.2)—the reference denotes the algorithm selected by `HashAlgo`. References to HKDF-BLAKE3 (Section 10.4) are not subject to `HashAlgo` and always denote BLAKE3.
Scope of hashing. Hashes cover raw on-disk bytes (compressed form if applicable). When `Encrypted` is set, all `PrevHash` fields **MUST** be zero (Section 10.6).

4.5 End-of-Capture Block (`0x0FFF`)

Completeness marker. An acquirer **SHOULD** emit it as the final block of the capture segment. A verified EoC block **MAY** be followed by extension segments (Section 4.6); bytes after the last verified segment are non-MSL trailing data.

Table 13: End-of-Capture payload (48 bytes).

Off	Size	Field	Description
+0x00	32	FileHash	H (HashAlgo) of all bytes from offset 0 through the last byte before this EoC block. Populated even when encrypted (computed over plaintext). Bytes 0x78–0x7F (CipherLen) are hashed as zero (Section 10.7).
+0x20	8	AcqEnd	Acquisition end time (UTC, ns since epoch).
+0x28	4	BlockCount	Number of capture-segment blocks including this EoC block. 0 = unknown. A producer **SHOULD** record the exact count here; it is always known when the EoC block is written (Section 3.1, *BlockCount semantics*).
+0x2C	4	Reserved	Zero.

Version 1.0.0 defined bytes +0x28–+0x2F as reserved (zero), which reads as BlockCount = 0; some 1.0.0 producers wrote a 40-byte payload without them. Consumers **MUST** accept a 40-byte EoC payload and treat its count as unknown.

4.6 Extension Segments and End-of-Extension Block (0x0FFE)

An MSL file consists of the file header, exactly one **capture segment**, and zero or more **extension segments**, in this order (Figure 6). The capture segment is the evidence: everything from Block 0 through the End-of-Capture block, exactly as the acquirer or importer wrote it. An extension segment is a run of one or more blocks terminated by an End-of-Extension block (0x0FFE). Extension segments let producers other than the acquirer—enrichment tools, analysis frameworks, importers preserving third-party annotations—add blocks after the capture has completed, without rewriting or re-hashing the capture segment and without patching the header. Each segment is self-delimiting and self-verifying, so a file remains usable if an appending tool is interrupted.

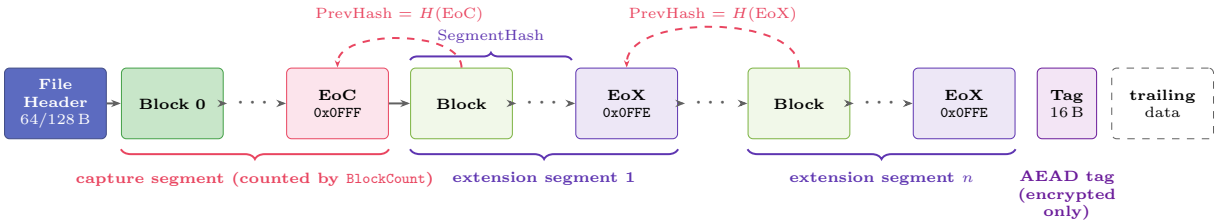


Figure 6: File layout with extension segments. Each segment is a run of blocks closed by an End-of-Extension (EoX) block whose SegmentHash covers the segment's preceding bytes; the PrevHash chain runs unbroken from the file header through every segment. In encrypted containers the segments lie inside the AEAD ciphertext and the tag follows the last segment (Section 10.2); sealed extension segments (Section 10.8) may follow the tag.

Chain continuity. The integrity chain continues across the End-of-Capture block: the PrevHash of the first block of the first extension segment is H of the EoC block's bytes, and the chain remains linear across segment boundaries (the first block of segment $k{+}1$ hashes the End-of-Extension block of segment k). When Encrypted is set, all PrevHash fields in extension segments **MUST** be zero, as in the capture segment.

Table 14: End-of-Extension payload.

Off	Size	Field	Abbr	Description
+0x00	32	SegmentHash	SegH	*H* (HashAlgo) of all bytes from the start of this segment's first block through the last byte before this End-of-Extension block.
+0x20	8	WrittenAt	WrAt	Time the segment was written (UTC, ns since epoch). 0 if unknown.
+0x28	4	BlockCount	BCt	Number of blocks in this segment, including this End-of-Extension block. **MUST** be exact (never 0).
+0x2C	4	Reserved	R	Zero.
+0x30	2	ProducerLen	PL	Length of Producer incl. NUL. 0 = none.
+0x32	6	Reserved2	R	Zero.
+0x38	var	Producer	—	UTF-8, pad8. Name and version of the appending tool (e.g., memslicer-enrich/0.5.0). Omitted when ProducerLen=0.

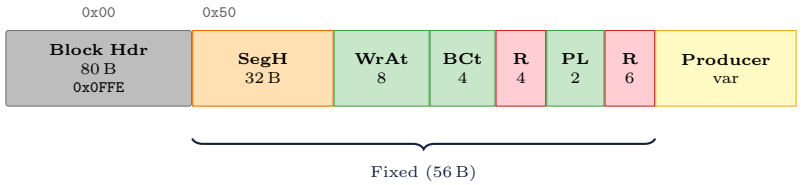


Figure 7: End-of-Extension (0x0FFE). Abbreviations per Table 14.

Permitted block types. Extension segments carry derived or supplementary data, never evidence. The following types **MUST NOT** appear in an extension segment: Memory Region (0x0001), Module Entry (0x0002), 0x0010–0x0015, Import Provenance (0x0030), Process Identity (0x0040), 0x0050–0x0058, and End-of-Capture (0x0FFF). Permitted types are Key Hint (0x0020), Related Dump (0x0041), 0x0059–0x005B (Sections 5.9–5.11), and the Structural namespace (0x1000–0x1FFF). Block types whose payload this specification does not define (e.g., Pointer Graph 0x1003, producer-defined Structural types) **MAY** appear only in extension segments. Blocks in an extension segment **MAY** reference blocks in the capture segment or in earlier segments by ParentUUID or payload-embedded UUID (Section 8).

Producer rules. A producer appending an extension segment **MUST NOT** modify any byte of the header, the capture segment, or an existing segment; **MUST** terminate every segment it writes with an End-of-Extension block; **MUST** continue the PrevHash chain from the last block already in the file (zero when encrypted); and **MUST** write the exact BlockCount and SegmentHash. A producer that finds trailing data after the last verified segment **SHOULD** refuse to append rather than build on unverified bytes. In an encrypted container, extension segments written before sealing **MUST** lie inside the primary AEAD ciphertext, i.e., be written before the AEAD is finalized; after sealing, a producer **MUST** append only sealed extension segments (Section 10.8).

Consumer rules. After parsing and verifying the EoC block (and, when encrypted, the AEAD tag), a consumer **MUST** attempt to parse extension segments: read blocks until an End-of-Extension block, then verify (a) chain continuity from the preceding block (unencrypted files), (b) SegmentHash, and (c) BlockCount. Only a segment that passes all three checks is a

verified segment; its blocks **MAY** then be presented alongside the capture. A segment that fails verification or is not terminated—EOF, non-magic bytes, or a block that fails framing before an End-of-Extension block is found—**MUST NOT** be presented as verified, and everything from its first byte to EOF is trailing data. Consumers that do not implement extension segments **MUST** treat everything after the EoC block as trailing data (consumer rule 14, Section 14). In an encrypted container, bytes after the primary AEAD tag are parsed as sealed extension segments (Section 10.8) before the remainder is treated as trailing data.

The next sections specify payload contents: per-process payloads (Section 5), investigation-mode payloads (Section 6), and the four-state page model (Section 7).

5 Capture-Time Payloads

Payload data begins at byte `0x50`. All offsets below are **relative to the payload start**.

5.1 Memory Region (`0x0001`)

Table 15: Memory Region payload.

Off	Size	Field	Abbr	Description
+0x00	8	BaseAddr	—	Virtual address of first byte.
+0x08	8	RegionSize	RegnSz	**MUST** be a multiple of PageSize.
+0x10	1	Protection	Pr	Bit 0=R, 1=W, 2=X, 3=Guard, 4=CoW.
+0x11	1	RegionType	RT	00=Unknown, 01=Heap, 02=Stack, 03=Image, 04=MappedFile, 05=Anonymous, 06=SharedMem, FF=Other.
+0x12	1	PageSizeLog2	PL	$2^{\texttt{PageSizeLog2}}$ bytes. Range $[10, 40]$.
+0x13	5	Reserved	R	Zero.
+0x18	8	Timestamp	Ts	Region acquisition time.
+0x20	var	PageStateMap	—	2 bits/page (Section 7). Pad to 8B.
$+m$	var	PageData	—	Captured pages only.

PageStateMap size. $P = \texttt{RegionSize} \gg \texttt{PageSizeLog2}$. Map: pad8$((P+3)/4)$ bytes.

Maximum region size and continuation blocks

The maximum payload per block is $2^{32} - 81$ bytes, derived from the 32-bit `BlockLength` (maximum value $2^{32} - 1$) minus the 80-byte Common Block Header. Regions exceeding this limit **MUST** be split into continuation blocks (`Continuation` flag set).

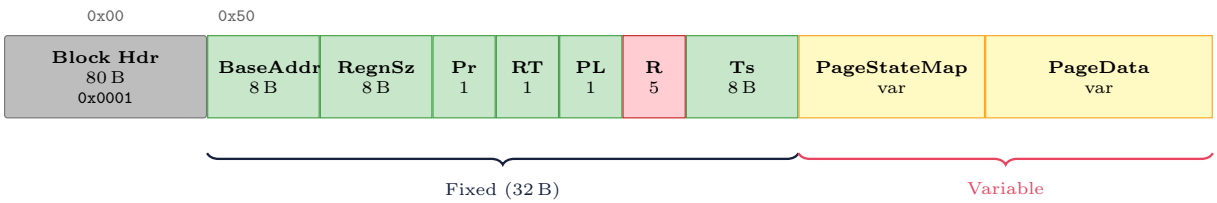


Figure 8: Memory Region (`0x0001`). Abbreviations per Table 15.

5.2 Module Entry (`0x0002`)

Stores the identity and metadata of a loaded library (DLL, .so): file path, version string, on-disk hash, and OS-native loader data. `ParentUUID` **MUST** reference the Module List Index (`0x0010`). `BlockUUID` **MUST** match the pre-assigned `ModuleUUID` from the manifest.

Table 16: Module Entry payload.

Off	Size	Field	Abbr	Description
+0x00	8	BaseAddr	—	Load address.
+0x08	8	ModuleSize	ModSz	Mapping size.
+0x10	2	PathLen	PL	Path length (incl. null, before padding).
+0x12	2	VersionLen	VL	Version length. 0 if unavailable.
+0x14	4	Reserved	R	Zero.
+0x18	var	Path	—	UTF-8, pad8.
$+p$	var	Version	—	UTF-8, pad8.
$+v$	32	DiskHash	—	H (HashAlgo, Section 4.4) of on-disk binary. 32 zero bytes if unavailable.
$+v$+32	4	BlobLen	BL	NativeBlob size.
$+v$+36	4	Reserved2	R	Zero.
$+v$+40	var	NativeBlob	—	OS-native opaque data.

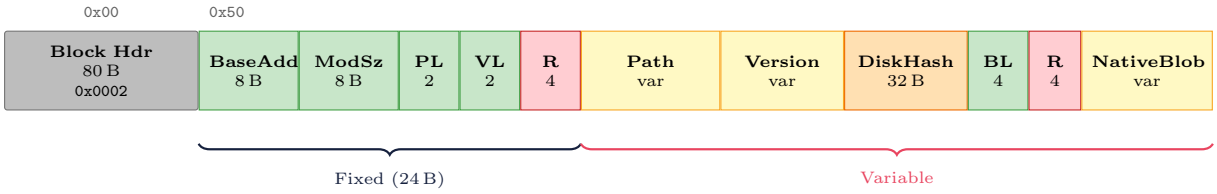


Figure 9: Module Entry (0x0002). Abbreviations per Table 16.

5.3 Module List Index (0x0010)

MUST be Block 1. Pre-generates UUIDv4 for each Module Entry block.

Table 17: Module List Index payload.

Off	Size	Field	Abbr	Description
— Preamble —				
+0x00	4	EntryCount	ECt	Number of module entries.
+0x04	4	Reserved	R	Zero.
— Entry (repeated ×EntryCount), offsets relative to entry start —				
+0x00	16	ModuleUUID	ModUUID	Pre-assigned BlockUUID.
+0x10	8	BaseAddr	—	Module load address.
+0x18	8	ModuleSize	ModSz	Mapping size.
+0x20	2	PathLen	PL	Path length (incl. null).
+0x22	2	Reserved	R	Zero.
+0x24	4	Reserved2	R	Zero.
+0x28	var	Path	—	UTF-8, pad8.

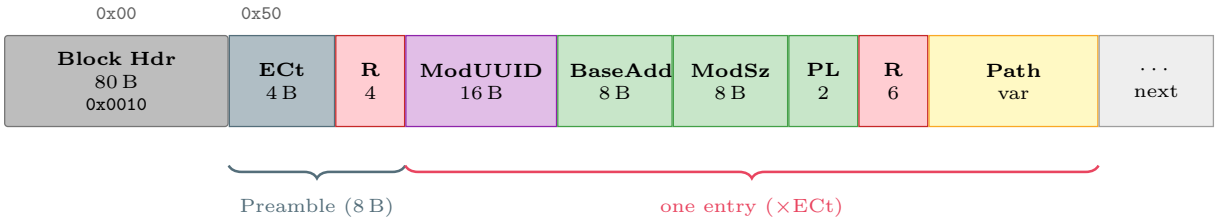


Figure 10: Module List Index (0x0010). Abbreviations per Table 17.

5.4 Process Identity (0x0040)

MUST be Block 0 for live acquisition (or after Import Provenance for imports).

Table 18: Process Identity payload.

Off	Size	Field	Abbr	Description
+0x00	4	PPID	—	Parent PID. 0 if unknown.
+0x04	4	SessionID	SID	0 if unknown.
+0x08	8	StartTime	StartTm	Process start time. 0 if unknown.
+0x10	2	ExePathLen	EPL	Incl. null.
+0x12	2	CmdLineLen	CLL	0 if unavailable.
+0x14	4	Reserved	R	Zero.
+0x18	var	ExePath	—	UTF-8, pad8.
$+e$	var	CmdLine	—	UTF-8, pad8.

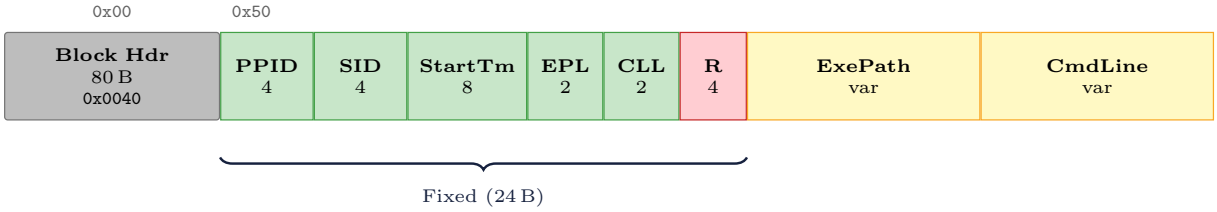


Figure 11: Process Identity (0x0040). Abbreviations per Table 18.

5.5 Related Dump (0x0041)

An acquirer **MAY** emit one or more Related Dump blocks to record relationships between concurrently captured processes (parent/child, shared-memory peers, IPC partners).

Table 19: Related Dump payload (24 bytes, fixed).

Off	Size	Field	Abbr	Description
+0x00	16	RelatedDumpUUID	RelDumpUUID	DumpUUID of related file.
+0x10	4	RelatedPID	RelPID	0 if unknown.
+0x14	2	Relationship	Rel	0x0001=Parent, 0x0002=Child, 0x0003=SharedMemory, 0x0004=IPC peer, 0x0005=Thread group, 0xFFFF=Other.
+0x16	2	Reserved	R	Zero.

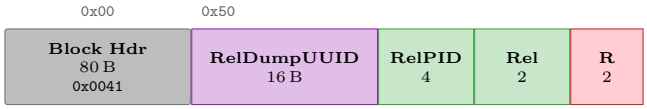


Figure 12: Related Dump (0x0041). Abbreviations per Table 19.

5.6 Key Hint (0x0020)

A Key Hint block annotates a region of captured memory that is believed to contain cryptographic key material. It records the key's location (via RegionUUID and byte offset), its type and protocol association, and a confidence level reflecting how the key was identified (speculative scan, heuristic match, or confirmed via instrumentation). Key Hints are **OPTIONAL**; their presence is indicated by the CryptoHints bit in CapBitmap and the HasKeyHints flag on the referenced Memory Region block.

Table 20: Key Hint payload.

Off	Size	Field	Abbr	Description
+0x00	16	RegionUUID	RgnUUID	Memory Region BlockUUID.
+0x10	8	RegionOffset	RgnOff	Byte offset in PageData.
+0x18	4	KeyLen	KLen	0 if unknown.
+0x1C	2	KeyType	KT	Table 21.
+0x1E	2	Protocol	Pr	Table 21.
+0x20	1	Confidence	C	0x00=Speculative, 0x01=Heuristic, 0x02=Confirmed.
+0x21	1	KeyState	S	0x00=Unknown, 0x01=Active, 0x02=Expired.
+0x22	2	Reserved	R	Zero.
+0x24	4	NoteLen	NL	0 if none.
+0x28	4	Reserved2	R	Zero.
+0x2C	var	Note	—	UTF-8, pad8.

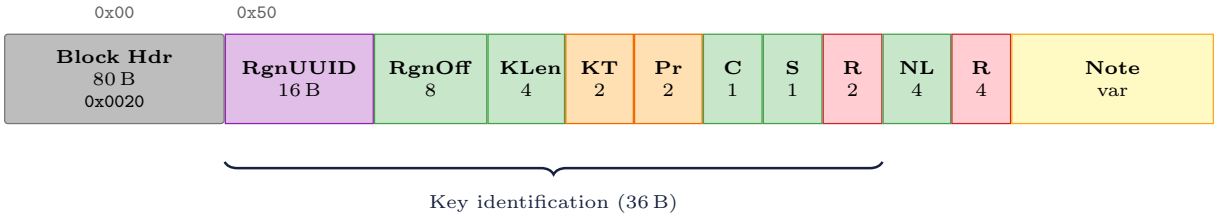


Figure 13: Key Hint (0x0020). Abbreviations per Table 20.

Table 21: Key type and protocol codes.

Key type codes (`KeyType`)

Code	Type
`0x0000`	Unknown
`0x0001`	Pre-Master Secret
`0x0002`	Master Secret
`0x0003`	Session Key
`0x0004`	Handshake Secret
`0x0005`	App Traffic Secret
`0x0006`	RSA Private Key
`0x0007`	ECDH Private Key
`0x0008`	IKE SA Key
`0x0009`	ESP/AH Key
`0x000A`	SSH Session Key
`0x000B`	WireGuard Key
`0x000C`	ML-KEM Private Key
`0xFFFF`	Other

Protocol codes (`Protocol`)

Code	Protocol
`0x0000`	Unknown
`0x0001`	TLS 1.2
`0x0002`	TLS 1.3
`0x0003`	DTLS 1.2
`0x0004`	DTLS 1.3
`0x0005`	QUIC
`0x0006`	IKEv2/IPsec
`0x0007`	SSH
`0x0008`	WireGuard
`0x0009`	PQ-TLS (hybrid)
`0xFFFF`	Other

5.7 Thread Context (`0x0011`)

A Thread Context block records the execution state of a single thread of the target process at acquisition time: the thread identifier, scheduling state, an optional thread name, and a register file. One block is emitted per captured thread. These blocks let a consumer reconstruct the CPU state required to emulate or single-step execution forward from the captured memory image. Thread Context blocks are **OPTIONAL**; their presence is advertised by the `ThreadContexts` bit (bit 2) in `CapBitmap`. `ParentUUID` **MAY** reference the Process Identity block (`0x0040`) or be zero.

Thread Context is an evidence block: it **MUST** be written in the capture segment (never in an extension segment, Section 4.6) and **SHOULD** precede the first Memory Region block so that consumers know the thread set before they classify regions (Section 6.1).

Exactly one Thread Context block in a file **SHOULD** set the `Current` flag, identifying the thread whose register file—in particular its program counter—represents the natural resumption point for emulation. If no block sets `Current`, consumers **SHOULD** treat the first Thread Context block in file order as current.

Table 22: Thread Context payload.

Off	Size	Field	Abbr	Description
+0x00	8	ThreadID	TID	OS thread identifier (TID). 0 if unknown.
+0x08	8	StartTime	StTm	Thread start time (UTC, ns since epoch). 0 if unknown.
+0x10	2	Flags	—	Bit 0=Current, bit 1=Crashed, bit 2=Suspended (registers were read from a stopped thread and are self-consistent), bit 3=Partial (register file incomplete). Bits 4–15 reserved.
+0x12	1	ThreadState	TS	0x00=Unknown, 0x01=Running, 0x02=Sleeping, 0x03=Stopped, 0x04=Waiting, 0x05=Uninterruptible, 0x06=Halted, 0x07=Zombie.
+0x13	1	Arch	Ar	0 = same as the file header ArchType; otherwise the ArchType code (Table 6b) of the mode this thread executes in (e.g., a WOW64 x86 thread in an x86_64 process, or an ARM64EC thread). Codes above 0xFF are not representable and **MUST** be written as 0.
+0x14	4	RegCount	RCt	Number of register entries that follow.
+0x18	2	NameLen	NL	Thread name length (incl. null). 0 if N/A.
+0x1A	6	Reserved2	R	Zero.
+0x20	var	ThreadName	—	UTF-8, pad8. Omitted when NameLen=0.
$+n$	var	Registers	—	RegCount register entries (Table 23).

Each register entry is self-describing and 8-byte aligned. The entry start (+0x00 below) is relative to the start of that entry.

Table 23: Thread Context register entry.

Off	Size	Field	Abbr	Description
+0x00	1	NameLen	NL	Register name length (incl. null).
+0x01	1	Width	W	Value width in bytes (e.g. 8 for GPRs; 16/32/64 for vector registers).
+0x02	2	Flags	—	Bit 0=program counter, bit 1=stack pointer, bit 2=frame pointer, bit 3=flags register, bit 4=thread pointer / TLS base, bit 5=link register. Bits 6–15 reserved.
+0x04	4	Reserved	R	Zero.
+0x08	var	Name	—	UTF-8, pad8. Lowercase canonical register mnemonic.
$+m$	var	Value	—	Width bytes. Integer registers in the file's byte order (Endianness, Section 1.3); vector registers in the target's native lane order. Pad to 8B.

Register names **MUST** be the lowercase canonical mnemonics of Table 24 for the architectures it lists. A producer that emits a Thread Context block **MUST** include at least the program-counter and stack-pointer registers, and **SHOULD** set the program-counter, stack-

pointer, frame-pointer, and flags bits in `Flags` on the corresponding entries so that consumers can locate them without architecture-specific knowledge. Consumers **MUST** preserve registers whose names they do not recognize and skip each entry via its declared `NameLen`/`Width` extents. **Canonical register names.** For the architectures listed in Table 24, register names **MUST** be the names given there. Aliases **MUST NOT** be written (ARM64 `fp`/`lr`; ARM32 `r13`–`r15`); consumers **SHOULD** normalise `fp`→`x29` and `lr`→`x30` in files from 1.1.0 producers. Registers not listed use the lowercase mnemonic of the architecture manual (e.g., `nzcv`, `v0`–`v31`, `xmm0`–`xmm15`). Listed integer registers, including flags and segment registers, use the GPR width, zero-extended. On x86_64, producers **MAY** write both `fs_base` and `gs_base`; the TLS flag (bit 4) is set only on the one the OS uses as thread pointer. Segment selectors (`fs`, `gs`) never carry the TLS flag; on x86 the TLS flag is set only if the producer records the segment base under a name not listed in Table 24 (e.g., `gs_base`).

Table 24: Canonical register mnemonics.

Arch	Names (role)
x86_64	`rax`, `rbx`, `rcx`, `rdx`, `rsi`, `rdi`, `rbp` (FP), `rsp` (SP), `r8`–`r15`, `rip` (PC), `rflags` (FLAGS), `cs`, `ss`, `ds`, `es`, `fs`, `gs`, `fs_base`/`gs_base` (TLS: the one the OS uses as thread pointer, `fs_base` on Linux/Android/FreeBSD, `gs_base` on Windows/macOS); optional `orig_rax`.
x86	`eax`, `ebx`, `ecx`, `edx`, `esi`, `edi`, `ebp` (FP), `esp` (SP), `eip` (PC), `eflags` (FLAGS), `cs`, `ss`, `ds`, `es`, `fs`, `gs`.
ARM64	`x0`–`x28`, `x29` (FP), `x30` (LR), `sp` (SP), `pc` (PC), `pstate` (FLAGS), `tpidr_el0` (TLS on Linux/Android/FreeBSD), `tpidrro_el0` (TLS on macOS/iOS), `x18` (TEB on Windows; TLS flag set on `x18`).
ARM32	`r0`–`r12`, `sp` (SP), `lr` (LR), `pc` (PC), `cpsr` (FLAGS), `tpidruro` (TLS); `r11` (FP in ARM mode), `r7` (FP in Thumb mode and on Apple platforms).

Importer mapping. ELF `NT_PRSTATUS`: x86_64 `user_regs_struct` field names map directly, with `eflags`→`rflags`; ARM64 `user_pt_regs` maps to `x0`–`x30`, `sp`, `pc`, `pstate`; ARM64 `tpidr_el0` comes from the `NT_ARM_TLS` note, not `NT_PRSTATUS`. Minidump `CONTEXT_AMD64`: `Rax`...`R15` and `Rip` map to the lowercase names, `EFlags`→`rflags`, and `SegCs`/`SegDs`/`SegEs`/`SegFs`/`SegGs`/`SegSs`→`cs`/`ds`/`es`/`fs`/`gs`/`ss`; it has no `fs_base`/`gs_base`, so importers **MAY** write `gs_base` from `MINIDUMP_THREAD.Teb` with the TLS flag. `CONTEXT_ARM64`: `X0`–`X28`→`x0`–`x28`, `Fp`→`x29`, `Lr`→`x30`, `Sp`→`sp`, `Pc`→`pc`, `Cpsr`→`pstate`; `x18` carries the TEB.

A producer **MUST** set `Suspended` only when the thread was stopped while its registers were read (e.g., ptrace-stopped, debugger-halted); registers sampled from a running thread **MUST** leave the bit clear, since they may not form a consistent snapshot. `Partial` **MUST** be set when the producer could not read every register it would normally record. Consumers **MAY** use the stack-pointer entries to attribute Memory Regions to threads (stack classification). Importers (Section 11) **MAY** emit Thread Context blocks from source-format thread records, such as ELF core `NT_PRSTATUS` notes or Minidump thread lists.

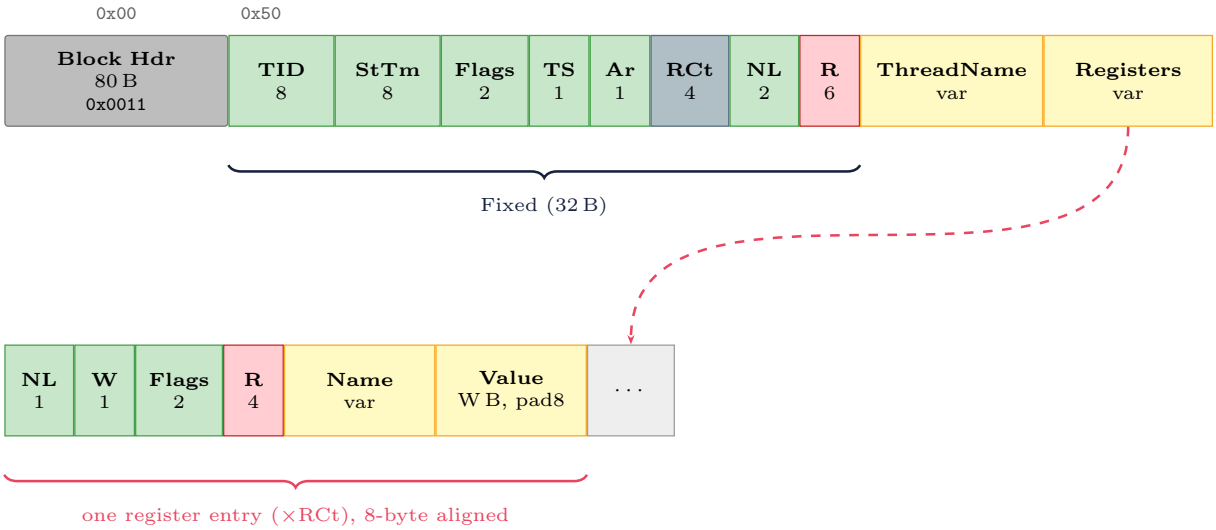


Figure 14: Thread Context (`0x0011`). Abbreviations per Tables 22 and 23.

5.8 VAS Map (`0x1001`)

A VAS Map records the target's enumerated virtual address space as a list of ranges with protection, type, and mapped path—one entry per range the acquirer enumerated, whether or not its bytes were captured. It is the only block from which a consumer can tell which parts of the address space existed but were not stored. Although it is a Structural-namespace block (Section 4.3), an acquirer **MAY** write it at capture time; the **RECOMMENDED** position is immediately before the End-of-Capture block, after all Memory Region and Module Entry blocks. An analysis tool **MAY** instead add it in an extension segment (Section 4.6). When a file contains several VAS Maps, the last one in file order supersedes earlier ones. `ParentUUID` is zero.

Table 25: VAS Map payload.

Off	Size	Field	Abbr	Description
— Preamble —				
+0x00	4	Count	Cnt	Number of entries.
+0x04	4	EntrySize	ESz	Size in bytes of every entry when all entries are the same size; 0 = variable (each entry sized by its PathLen).
— Entry (repeated ×Count), offsets relative to entry start —				
+0x00	8	BaseAddr	—	Virtual address of first byte.
+0x08	8	RegionSize	RegnSz	Range size in bytes.
+0x10	1	Protection	Pr	As Table 15.
+0x11	1	RegionType	RT	As Table 15.
+0x12	2	PathLen	PL	Mapped path length (incl. null). 0 if none.
+0x14	1	NameSource	NS	Origin of MappedPath and RegionType (below).
+0x15	3	Reserved	R	Zero.
+0x18	var	MappedPath	—	UTF-8, pad8. Omitted when PathLen=0.

`NameSource` codes. `0x00`=Unspecified (a consumer infers: an entry in the capture segment of a file without the `Imported` flag was enumerated live); `0x01`=Live (enumerated from the running

process by the acquirer); 0x02=SourceMetadata (taken from the metadata of an imported dump, e.g., ELF NT_FILE or Minidump module/memory-info streams); 0x03=Synthesized (derived by the producer, e.g., from captured region boundaries or heuristics). Files written before this revision carry zero at +0x14, which reads as Unspecified. Consumers **SHOULD** trust names only from Live or SourceMetadata entries when classifying regions.

Semantics. An entry whose BaseAddr has no Memory Region block of the same base means the range existed at enumeration time but its bytes were not stored (omitted by scope, excluded, or never attempted); the Region Omission Manifest (Section 5.11) says why. A Memory Region without a matching VAS Map entry is a range the enumerator did not report. Consumers **MUST NOT** derive page states from the VAS Map; the PageStateMap (Section 7) is authoritative for captured ranges.

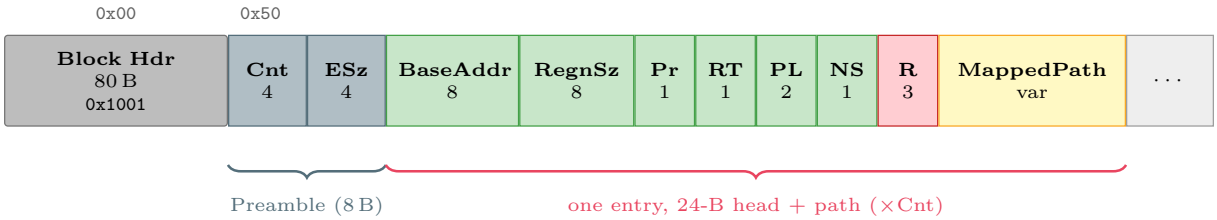


Figure 15: VAS Map (0x1001). Abbreviations per Table 25.

5.9 Physical Memory Map (0x0059)

A Physical Memory Map records the host's physical address ranges (e.g., the top-level entries of Linux /proc/iomem: "System RAM", "Reserved", PCI windows) at acquisition time. It gives analysis tools that correlate a process capture with a physical memory image the layout they need to translate addresses. The block is **OPTIONAL** and is defined here so that the type code and layout are stable; it is not yet emitted by any known producer. It **MAY** be written in the capture segment or in an extension segment (Section 4.6). ParentUUID is zero.

Table 26: Physical Memory Map payload.

Off	Size	Field	Abbr	Description
— Preamble —				
+0x00	4	RowCount	RC	Number of rows.
+0x04	4	Reserved	R	Zero.
— Row (repeated ×RowCount), offsets relative to row start —				
+0x00	8	Start	—	First physical address of the range.
+0x08	8	End	—	Last physical address of the range (inclusive, as reported by /proc/iomem).
+0x10	2	LabelLen	LL	Byte length of Label. **No** terminating NUL.
+0x12	2	Reserved	R	Zero.
+0x14	var	Label	—	UTF-8, exactly LabelLen bytes, **not** padded.

Rows are packed back to back and are therefore not 8-byte aligned; only the payload as a whole is padded to an 8-byte boundary. Consumers **MUST** walk rows by LabelLen.

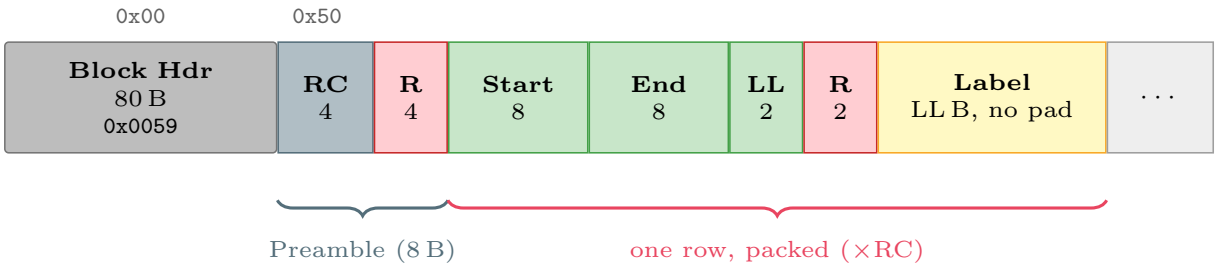


Figure 16: Physical Memory Map (`0x0059`). Abbreviations per Table 26.

5.10 Module Build-ID Manifest (`0x005A`)

A Module Build-ID Manifest supplements the Module Entry blocks of a capture with build identifiers (ELF `NT_GNU_BUILD_ID`, PE CodeView GUID+age, Mach-O `LC_UUID`) and on-disk hashes that were not available—or not yet computed—when the acquirer wrote the module list. It is written by enrichment tools, either in the capture segment or in an extension segment (Section 4.6). Readers merge rows into the module list by matching `BaseAddr` against Module Entry `BaseAddr`; a non-zero `DiskHash` in a row supersedes an all-zero `DiskHash` in the matching Module Entry. Rows whose `BaseAddr` matches no Module Entry **SHOULD** be reported as dangling (Section 8) but **MUST NOT** be discarded. `ParentUUID` is zero.

Table 27: Module Build-ID Manifest payload.

Off	Size	Field	Abbr	Description
— Preamble —				
+0x00	4	RowCount	RC	Number of 64-byte rows.
+0x04	4	Reserved	R	Zero.
— Row (repeated ×RowCount), 64 bytes fixed, offsets relative to row start —				
+0x00	8	BaseAddr	—	Module load address; join key to Module Entry / Module List Index.
+0x08	1	BuildIdLen	BL	Valid bytes in `BuildId`: 0 (none), 16, or 20.
+0x09	1	BuildIdSource	BS	How the build ID was obtained (below).
+0x0A	1	Flags	—	Bit 0=`Deleted` (backing file unlinked while mapped), bit 1=`MemFd` (anonymous `memfd` mapping), bit 2=`AnonRWX` (anonymous mapping with RWX protection), bit 3=`Unlinked` (path not found on disk). Bits 4–7 reserved.
+0x0B	1	Reserved	R	Zero.
+0x0C	20	BuildId	—	Build identifier, zero-padded to 20 bytes.
+0x20	32	DiskHash	—	*H* (`HashAlgo`) of the on-disk binary, zero-padded; 32 zero bytes if unavailable.

`BuildIdSource` codes. 0=None, 1=Bridge (reported by the instrumentation bridge, e.g., Frida/GDB/LLDB), 2=MapFiles (read from the mapped file via `/proc/pid/map_files`), 3=OnDisk (read from the file at its recorded path), 4=CapturedRegion (parsed from the captured image bytes), 5=Retroactive (computed after acquisition by an enrichment tool).

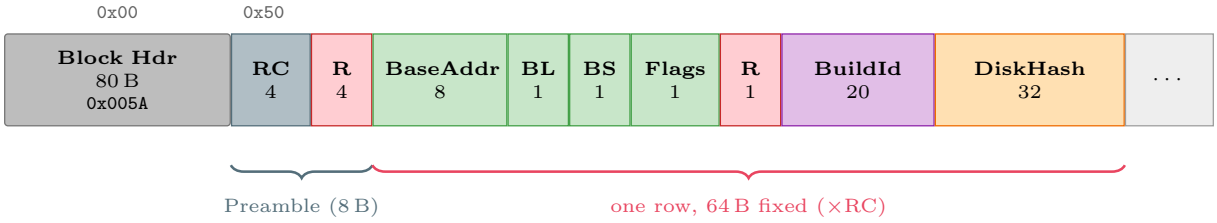


Figure 17: Module Build-ID Manifest (`0x005A`). Abbreviations per Table 27.

5.11 Region Omission Manifest (`0x005B`)

A Region Omission Manifest lists the address ranges that the acquirer *deliberately* did not capture—because of a region scope selection (e.g., heap and stack only) or module exclusion patterns—so that a consumer can distinguish a partial capture from an incomplete one. Omitted ranges are absent from the Memory Region stream but present in the VAS Map (Section 5.8); this block records the reason. A producer **MUST** emit it only when at least one range was omitted, and **MAY** write it in the capture segment or in an extension segment (Section 4.6). `ParentUUID` is zero.

Unlike the general convention of Section 1.3, the strings in this block carry **no terminating NUL**: every length field counts UTF-8 bytes only, and each string is zero-padded to an 8-byte boundary.

Table 28: Region Omission Manifest payload.

Off	Size	Field	Abbr	Description
— Preamble —				
+0x00	4	RowCount	RC	Number of rows.
+0x04	4	Reserved	R	Zero.
+0x08	2	ScopeLen	SL	Byte length of Scope (no NUL). 0 if none.
+0x0A	2	PatternsLen	PtL	Byte length of Patterns (no NUL). 0 if none.
+0x0C	4	Reserved2	R	Zero.
+0x10	var	Scope	—	UTF-8, pad8. The scope selection as given to the acquirer (e.g., heap,stack).
$+s$	var	Patterns	—	UTF-8, pad8. Exclusion patterns joined with \n (0x0A).
— Row (repeated ×RowCount), 24-byte head + name, offsets relative to row start —				
+0x00	8	BaseAddr	—	Virtual address of first byte of the omitted range.
+0x08	8	Size	—	Range size in bytes.
+0x10	1	Protection	Pr	As Table 15.
+0x11	1	RegionType	RT	As Table 15.
+0x12	1	Reason	Rsn	1=ModuleExclude (matched an exclusion pattern), 2=Scope (outside the selected region scope). 3–255 reserved.
+0x13	1	Reserved	R	Zero.
+0x14	2	NameLen	NL	Byte length of Name (no NUL). 0 if none.
+0x16	2	Reserved2	R	Zero.
+0x18	var	Name	—	UTF-8, pad8. Mapped path or region name. Omitted when NameLen=0.

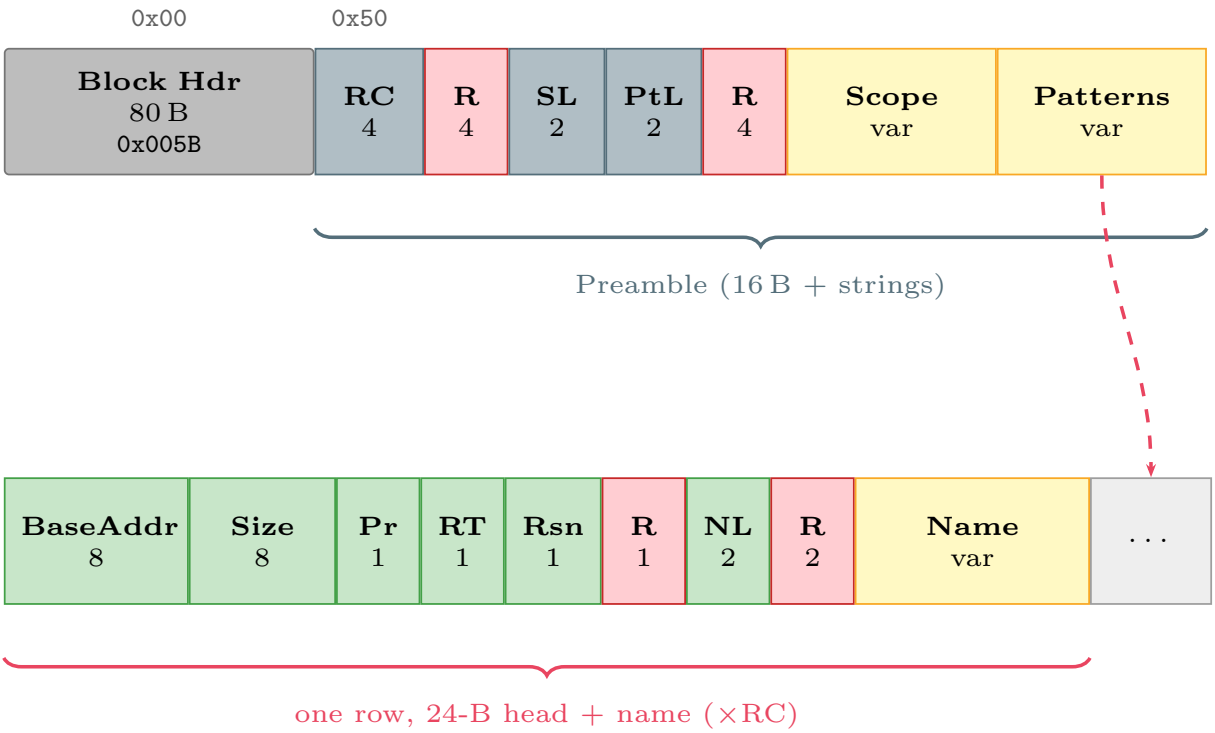


Figure 18: Region Omission Manifest (`0x005B`). Abbreviations per Table 28.

6 Investigation Mode

When `Investigation` (bit 1) is set, the acquirer **MUST** capture system-wide context alongside per-process memory: acquisition operator, system hostname, and system-wide tables (process list, network connections, open handles). When not set, the acquisition is in Analysis mode and **MUST** follow Figure 2.

6.1 Block Ordering

1. **Block 0:** Process Identity (`0x0040`).
2. **Block 1:** Module List Index (`0x0010`).
3. **Block 2:** System Context (`0x0050`) — **REQUIRED**.

System-wide table blocks (`0x0051`–`0x0053`) follow with `ParentUUID` referencing the System Context block; a Connectivity Table (`0x0054`, Section 6.6) **MAY** follow them with `ParentUUID` zero. Thread Context blocks (`0x0011`), when present, **SHOULD** precede the first Memory Region block in either mode (Section 5.7, Figure 2).

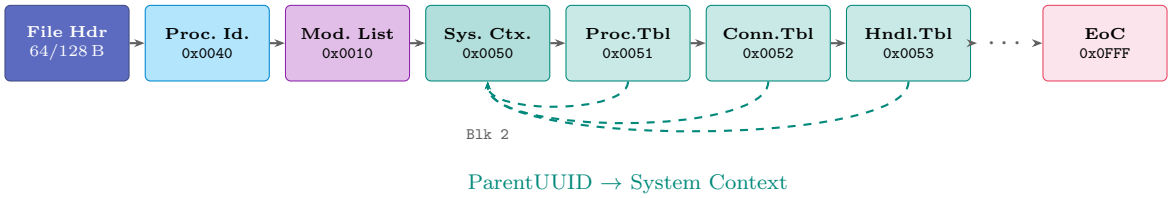


Figure 19: Investigation mode block sequence. System table blocks reference System Context via `ParentUUID`.

6.2 System Context Block (`0x0050`)

The System Context block captures system-wide metadata at acquisition time: the identity of the acquisition operator, the target system's hostname and OS version, and a bitmap indicating which system-wide table blocks (`0x0051`–`0x0054`) are present. This block **MUST** be Block 2 when `Investigation` is set. The `TableBitmap` allows consumers to determine which tables to expect without scanning ahead.

Table 29: System Context payload.

Off	Size	Field	Abbr	Description
+0x00	8	BootTime	BootTm	System boot time. 0 if unknown.
+0x08	4	TargetCount	TCt	Processes targeted this session.
+0x0C	4	TableBitmap	TBm	Bit 0=ProcessTable, 1=ConnectionTable, 2=HandleTable, 3=ConnectivityTable (Table 30).
+0x10	2	AcqUserLen	—	Incl. null.
+0x12	2	HostnameLen	—	Incl. null.
+0x14	2	DomainLen	—	0 if N/A.
+0x16	2	OSDetailLen	—	
+0x18	2	CaseRefLen	—	0 if none.
+0x1A	6	Reserved	R	Zero.
+0x20	var	AcqUser	—	UTF-8, pad8. Acquisition operator.
$+u$	var	Hostname	—	UTF-8, pad8.
$+h$	var	Domain	—	UTF-8, pad8. Omitted if len=0.
$+d$	var	OSDetail	OSDet	UTF-8, pad8.
$+o$	var	CaseRef	—	UTF-8, pad8. Omitted if len=0.

The TableBitmap field advertises which tables and system-context extension blocks are present in the slice. Each bit is associated with a single block identifier, so that a reader can derive the expected block inventory directly from the System Context block rather than by scanning the slice. Bit assignments are listed in Table 30; bits 4–31 are reserved and **MUST** be emitted as zero.

Table 30: TableBitmap bit assignments.

Bit	Mask	Block	Set when
0	0x0001	ProcessTable (0x0051)	The system process table is emitted.
1	0x0002	ConnectionTable (0x0052)	The system socket-connection table is emitted.
2	0x0004	HandleTable (0x0053)	The per-target handle table is emitted.
3	0x0008	ConnectivityTable (0x0054)	The collector produced at least one row of kernel network state (routes, ARP, packet sockets, interface counters, socket-family aggregates, or MIB counters).

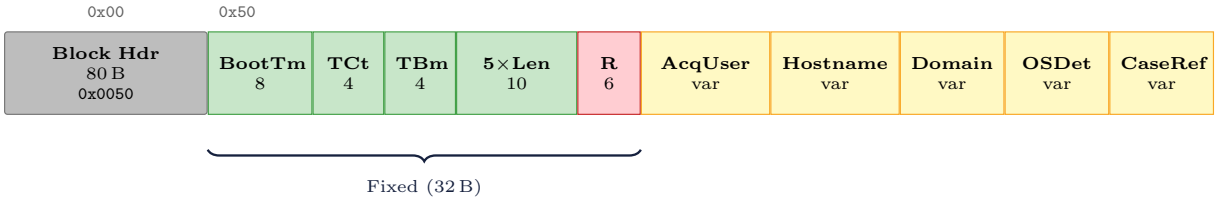


Figure 20: System Context (0x0050). Abbreviations per Table 29.

The block definitions that follow specify the system-context metadata classes in the identifier range 0x0051–0x0054. Each block is additive: readers that do not recognize a block type skip it according to the rules of Section 4, and the format therefore remains forward compatible. Presence is advertised through a dedicated bit in the System Context TableBitmap (Table 30).

6.3 Process Table (0x0051)

System-wide process snapshot. ParentUUID **MUST** reference the System Context block.

Table 31: Process Table payload.

Off	Size	Field	Abbr	Description
— Preamble —				
+0x00	4	EntryCount	ECt	Number of process entries.
+0x04	4	Reserved	R	Zero.
— Entry (repeated ×EntryCount), offsets relative to entry start —				
+0x00	4	PID	—	
+0x04	4	PPID	—	Parent PID.
+0x08	4	UID	—	POSIX UID or SID hash.
+0x0C	1	IsTarget	T	0x01 if acquisition target.
+0x0D	3	Reserved	R	Zero.
+0x10	8	StartTime	StTm	0 if unknown.
+0x18	8	RSS	—	0 if unknown.
+0x20	2	ExeNameLen	—	Incl. null.
+0x22	2	CmdLineLen	—	0 if unavailable.
+0x24	2	UserLen	—	0 if unavailable.
+0x26	2	Reserved2	R	Zero.
+0x28	var	ExeName	—	UTF-8, pad8.
$+e$	var	CmdLine	—	UTF-8, pad8.
$+c$	var	User	—	UTF-8, pad8.

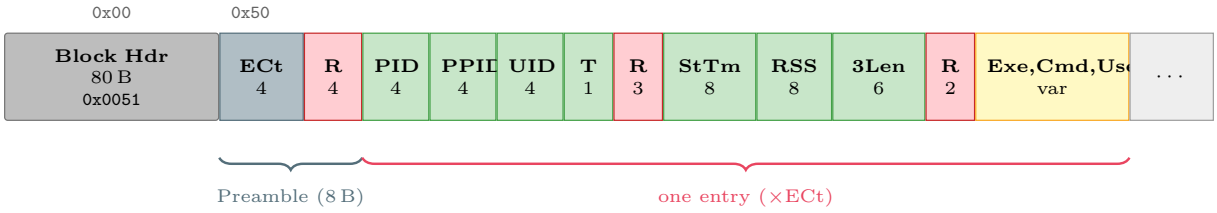


Figure 21: Process Table (0x0051). Abbreviations per Table 31.

6.4 Connection Table (0x0052)

System-wide network sockets. `ParentUUID` **MUST** reference System Context.

Port byte order

`LocalPort` and `RemotePort` are structural integers and therefore stored as little-endian `uint16`, regardless of the network byte order (big-endian) used by OS socket APIs. Producers **MUST** convert from network byte order before writing.

Table 32: Connection Table payload.

Off	Size	Field	Abbr	Description
— Preamble —				
+0x00	4	EntryCount	ECt	Number of connection entries.
+0x04	4	Reserved	R	Zero.
— Entry (repeated ×EntryCount), 48 bytes fixed, offsets relative to entry start —				
+0x00	4	PID	—	0 if unknown.
+0x04	1	Family	F	0x02=IPv4, 0x0A=IPv6.
+0x05	1	Protocol	P	0x06=TCP, 0x11=UDP.
+0x06	1	State	S	TCP state. 0x00=N/A.
+0x07	1	Reserved	R	Zero.
+0x08	16	LocalAddr	—	IPv4: 4B used, rest zero.
+0x18	2	LocalPort	LP	Little-endian uint16.
+0x1A	2	Reserved2	R	Zero.
+0x1C	16	RemoteAddr	—	Zeros for LISTEN.
+0x2C	2	RemotePort	RP	0 for LISTEN/UDP.
+0x2E	2	Reserved3	R	Zero.

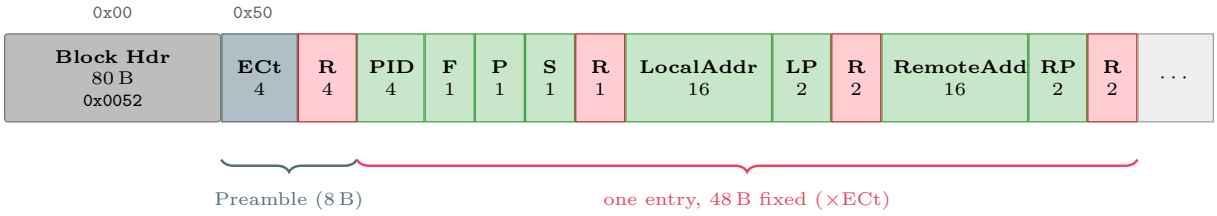


Figure 22: Connection Table (0x0052). Abbreviations per Table 32.

6.5 Handle Table (0x0053)

System-wide file descriptors/handles. `ParentUUID` **MUST** reference System Context.

Table 33: Handle Table payload.

Off	Size	Field	Abbr	Description
— Preamble —				
+0x00	4	EntryCount	ECt	Number of handle entries.
+0x04	4	Reserved	R	Zero.
— Entry (repeated ×EntryCount), offsets relative to entry start —				
+0x00	4	PID	—	
+0x04	4	FD	—	Descriptor/handle number.
+0x08	1	HandleType	HT	0x00=Unknown, 0x01=File, 0x02=Dir, 0x03=Socket, 0x04=Pipe, 0x05=Device, 0x06=Registry, 0xFF=Other.
+0x09	1	Reserved	R	Zero.
+0x0A	2	PathLen	PL	0 if unavailable.
+0x0C	4	Reserved2	R	Zero.
+0x10	var	Path	—	UTF-8, pad8.

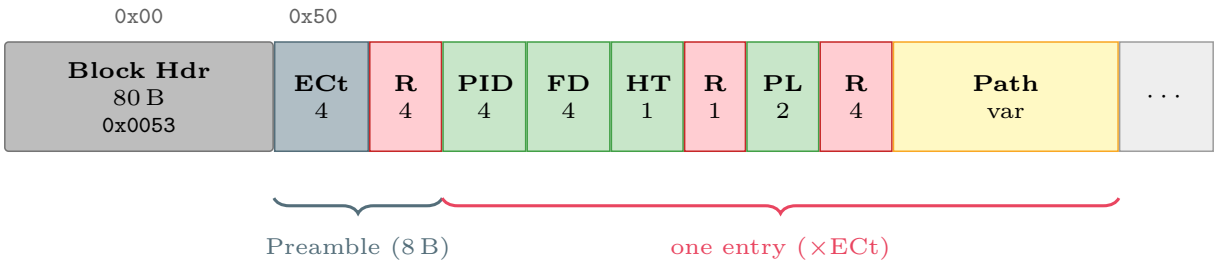


Figure 23: Handle Table (0x0053). Abbreviations per Table 33.

6.6 Connectivity Table (0x0054)

The Connectivity Table records kernel network state that has no socket-endpoint representation and therefore does not fit the schema of the Connection Table block (0x0052). The two blocks are complementary: endpoints with process attribution remain in the Connection Table, while routing entries, neighbour-discovery (ARP) caches, raw packet sockets, per-interface counters, aggregate socket-family statistics, and Management Information Base (MIB) counters are recorded here. The separation allows downstream analyzers to distinguish the two forensic questions of *who* was connected to *what* and *what was the network state at capture time*. The latter is essential for detecting raw-socket observers whose presence is invisible to the conventional socket table, because they possess neither an address nor a port tuple.

The block is emitted only in Investigation mode; its presence is advertised by TableBitmap bit 3 (Table 30). Unlike 0x0051–0x0053, ParentUUID is zero. The payload is a heterogeneous tagged-row table. Each row begins with a one-byte row-type discriminator, followed by a two-byte row length and a row-type-specific body. Seven row shapes are defined; readers that do not recognize a row type advance by RowLen bytes.

Table 34: Connectivity Table payload.

Off	Size	Field	Abbr	Description
+0x00	4	RowCount	RC	Total number of rows across all row types.
+0x04	4	Reserved	R	Zero.
+0x08	var	Rows	—	Concatenation of tagged rows (see Table 35).

Each row is prefixed by `RowType(1)` `RowLen(2)` and followed by `RowLen` bytes of row-specific body (`RowLen` excludes the 3-byte prefix). Rows are packed back to back without internal padding; only the payload as a whole is padded to an 8-byte boundary. The defined row types are enumerated in Table 35. String-typed fields within row bodies use a two-byte length prefix followed by `Len` bytes of UTF-8 content (no NUL). All multi-byte integers are little-endian. IPv4 addresses are carried as four raw bytes in network order; IPv6 addresses as sixteen raw bytes. Row `0x07` stores `Value` as an unsigned 8-byte field; counters the kernel reports as signed (e.g., `Tcp.MaxConn` = −1) are written in two's complement so that the evidence survives verbatim.

Table 35: Connectivity Table row-type discriminators.

Type	Name	Body schema
0x01	IPv4 route	`IfaceLen(2), Iface, Dest(4), Gateway(4), Mask(4), Flags(2), Metric(4), Mtu(4)`
0x02	IPv6 route	`IfaceLen(2), Iface, Dest(16), DestPrefix(1), NextHop(16), Metric(4), Flags(4)`
0x03	ARP entry	`Family(1), Ip(4), HwType(2), Flags(2), HwAddr(6), IfaceLen(2), Iface`
0x04	Packet socket	`Pid(4), Inode(8), Proto(2), IfaceIndex(4), User(4), Rmem(8)`
0x05	Interface statistics	`IfaceLen(2), Iface, RxBytes(8), RxPkts(8), RxErr(8), RxDrop(8), TxBytes(8), TxPkts(8), TxErr(8), TxDrop(8)`
0x06	Socket-family aggregate	`Family(1), InUse(4), Alloc(4), Mem(8)`
0x07	MIB counter	`MibLen(2), Mib, CounterLen(2), Counter, Value(8)`

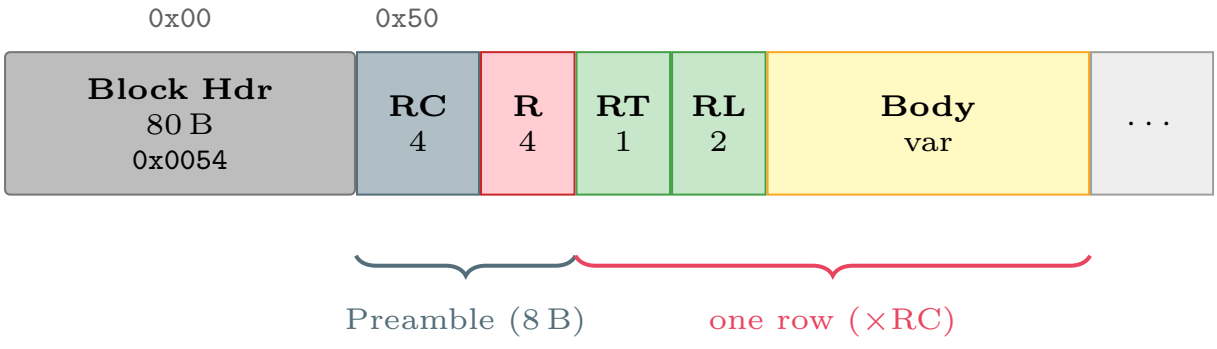


Figure 24: Connectivity Table (`0x0054`). Abbreviations per Tables 34 and 35.

6.7 Redaction and Data Minimization

Investigation-mode fields such as `AcqUser`, `Hostname`, `Domain`, and `CaseRef` may contain personally identifiable information or data subject to disclosure restrictions. This section provides guidance for producers that must balance forensic completeness with privacy requirements.

- Producers **MAY** emit zero-length strings (Len=0) for `AcqUser`, `Domain`, and `CaseRef` when organizational policy requires data minimization. A conformant consumer **MUST** handle zero-length optional strings gracefully.
- When any field has been intentionally stripped or pseudonymized, producers **MUST** set the `Redacted` flag (bit 3) in the file header. This distinguishes intentional redaction from fields that were merely unavailable at acquisition time: a zero-length string with `Redacted`=0 means "not captured"; the same string with `Redacted`=1 means "captured but removed."
- When sharing MSL files for research or cross-organizational analysis, producers **SHOULD** strip or pseudonymize personally identifiable fields prior to distribution and set the `Redacted` flag.
- `Hostname` and `OSDetail` carry forensic value that may be essential for investigation continuity. Redaction of these fields **SHOULD** be documented in the case log.

7 Page State Map

Each page within a Memory Region is classified into one of four states, encoded as 2 bits in the `PageStateMap`. This model enforces *epistemic honesty*: rather than silently omitting pages that could not be read, the format distinguishes successful capture from read failures and TOCTOU (Time-of-check to time-of-use) races (pages that were mapped when the region was enumerated but absent when actually read). Table 36 defines the states; Figure 25 shows an example map.

Table 36: Page states.

Bits	State	Meaning	PageData
00	Captured	Read OK; stored.	PageSize bytes
01	Failed	Mapped but unreadable.	0
10	Unmapped	TOCTOU: absent at read time.	0
11	NotResident	Mapped but never touched at read time (anonymous memory, not present and not swapped); content is zero. Emitted only by resident-only acquisition.	0

A NotResident page carries no data exactly like Failed and Unmapped. Readers that predate this state **MUST** treat it as a 0-byte page; treating it as Failed is acceptable but overstates loss, since the page's content is known to be zero.

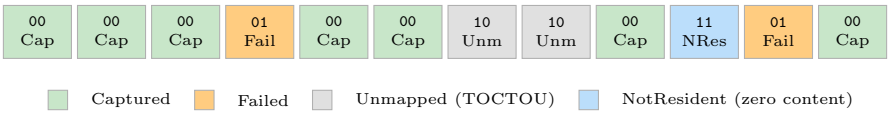


Figure 25: Example PageStateMap for a 12-page region.

8 Block Cross-Referencing

Layer 1: `ParentUUID`**.** Header-level hierarchy. Module Entry → Module List; system tables → System Context.

Layer 2: Payload-embedded UUIDs. Key Hint `RegionUUID` → Memory Region.

Address-keyed joins. VAS Map entries (Section 5.8), Module Build-ID Manifest rows (Section 5.10), and Region Omission Manifest rows (Section 5.11) correlate with Memory Region and Module Entry blocks by `BaseAddr`, not by UUID.

Across segments. Cross-references span segments: a block in an extension segment **MAY** reference blocks in the capture segment or in earlier extension segments (Section 4.6). Blocks in the capture segment never reference extension blocks, which are written later.

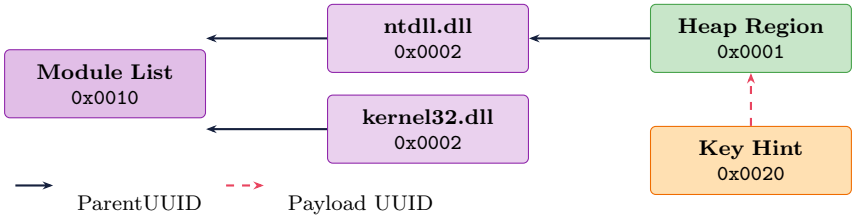


Figure 26: Block cross-referencing.

Dangling references. Consumers encountering a `ParentUUID` or payload-embedded UUID that does not match any block in the file **SHOULD** log the dangling reference and continue parsing. The block with the unresolvable reference **SHOULD** be treated as an orphan but **MUST NOT** be discarded, as the referenced block may have been lost to corruption rather than being absent by design.

9 Capability Bitmap

The 64-bit `CapBitmap` in the file header advertises which categories of data the file contains. Consumers can inspect the bitmap before parsing any blocks to determine whether the information they need (e.g., module metadata, crypto key hints, system-wide tables) was captured. Each bit corresponds to one or more block types as shown in Table 37. Producers **MUST** set bits accurately; consumers **SHOULD** use the bitmap to report missing data categories rather than silently producing incomplete results.

Table 37: Capability bits (64-bit `CapBitmap`).

Bit	Name	Description
0	MemoryRegions	`0x0001`.
1	ModuleList	`0x0010`/`0x0002`.
2	ThreadContexts	`0x0011`.
3	FileDescriptors	`0x0012`.
4	NetworkState	`0x0013`.
5	EnvironmentVars	`0x0014`.
6	SharedMemory	IPC identifiers.
7	SecurityContext	`0x0015`.
8	ProcessIdentity	`0x0040`.
9	RelatedDumps	`0x0041`.
10	CryptoHints	`0x0020`.
11	SystemContext	`0x0050`. **MUST** be set when `Investigation` set.
12	SystemProcessTable	`0x0051`.
13	SystemNetworkTable	`0x0052`.
14	SystemHandleTable	`0x0053`.
15–63	Reserved	Zero.

10 Encryption

10.1 Scope

When the `Encrypted` flag (bit 2) is set, the entire block stream is protected by AEAD (Authenticated Encryption with Associated Data). The file header remains in cleartext for format identification. All block data—including block types, UUIDs, timestamps, module paths, and memory content—is encrypted.

Full-container encryption is chosen over per-block encryption to prevent metadata leakage: per-block schemes would expose block types, block counts, block sizes, and timing patterns, all of which can reveal forensically sensitive information (e.g., which libraries are loaded, how many memory regions exist) without the decryption key.

10.2 Encrypted File Layout

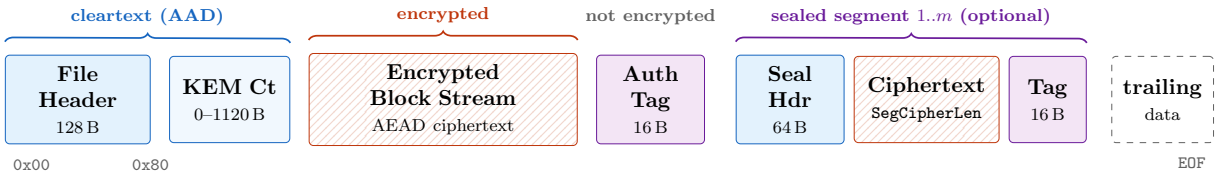


Figure 27: Encrypted file layout. The AEAD encrypts the block stream; the tag is an output, not itself encrypted. When `CipherLen`≠0, sealed extension segments (Section 10.8) and then trailing data may follow the tag.

1. **File header (128 B, cleartext):** Authenticated as Associated Data (AAD) to the AEAD. The AAD is the file header with bytes `0x78`–`0x7F` (`CipherLen`) set to zero, followed by any KEM ciphertext. Tampering causes decryption failure.
2. **KEM ciphertext (0 or variable, cleartext):** Present when `KeyEncap`≠`0x00`. Size is `KEMCtLen`. Also in AAD (item 1).
3. **Encrypted block stream:** AEAD ciphertext of Block 0 through EoC, followed by any extension segments (Section 4.6). Its length is `CipherLen` when non-zero. The only encrypted region of the primary AEAD.
4. **AEAD authentication tag (16 B, not encrypted):** Output of the AEAD. Required for decryption. Requires the symmetric key. Immediately follows the ciphertext.
5. **Sealed extension segments (optional, cleartext header + ciphertext + tag):** Only when `CipherLen`≠0 (Section 10.8).

Locating the tag. When `CipherLen`=0 (all 1.1.0 files, and non-seekable streaming producers, Section 10.7), the tag is the final 16 bytes of the file; such a file cannot carry sealed segments or trailing data. When `CipherLen`≠0, the ciphertext occupies [`HeaderSize`+`KEMCtLen`, `HeaderSize`+`KEMCtLen`+`CipherLen`) and the tag immediately follows it; sealed extension segments (Section 10.8), then trailing data, follow the tag. Consumers **MUST** reject `CipherLen` > file size − `HeaderSize` − `KEMCtLen` − 16. `CipherLen` lies outside the AAD, but changing it moves the tag boundary and thus changes the authenticated ciphertext and the tag bytes; verification then fails except with negligible probability. A consumer that fails tag verification on a file whose bytes `0x78`–`0x7F` are non-zero **SHOULD** report that the file may require 1.1.1 support rather than reporting tampering. For `CipherLen`=0 the AAD is byte-identical to the 1.1.0 AAD.

Extension segments in encrypted containers. Extension segments written before sealing **MUST** lie inside the primary AEAD ciphertext, i.e., they are written before the AEAD is finalized, and the primary tag covers them. Enrichment after sealing appends sealed extension segments (Section 10.8), each separately authenticated and chained to the preceding tag; it requires the CEK and `CipherLen`≠0. Bytes after the primary tag that are not verified sealed segments are trailing non-MSL data.

Chain-of-custody without the key. Producers **SHOULD** record an external hash of the complete file at acquisition time (BLAKE3 or SHA-256, offset 0 to EOF), computed after the `CipherLen` patch, i.e., over the final file. This digest **SHOULD** be stored in the case evidence log, not in the file.

10.3 Cipher Suites

Table 38: AEAD cipher suites.

Code	Suite	Nonce	Tag	Notes
0x01	AES-256-GCM	12 B	16 B	Default. Fast with AES-NI.
0x02	XChaCha20-Poly1305	24 B	16 B	Safe random nonce.

Both ciphers provide 256-bit keys. AES-256 and ChaCha20 are post-quantum safe at the symmetric level (Grover reduces effective key strength by half: AES-256 → 128-bit equivalent). The post-quantum vulnerability lies in key *encapsulation*, addressed below.

10.4 Key Encapsulation

The `KeyEncap` field specifies how the content encryption key (CEK) was protected. After KEM decapsulation, the shared secret is passed through HKDF-BLAKE3 (salt=`DumpUUID`, info=`"MSL-CEK-v1"`) to derive the 256-bit CEK.

Table 39: Key encapsulation mechanisms.

Code	Mechanism	Ct Size	Notes
0x00	None	0	Raw key (passphrase or external).
0x01	X25519	32 B	Classical ECDH.
0x02	ML-KEM-768	1088 B	FIPS 203 [7]. NIST PQ standard.
0x03	ML-KEM-1024	1568 B	FIPS 203. Higher margin.
0x04	X25519+ML-KEM-768	1120 B	Hybrid. **Recommended.**
0x05--0xFE	*Reserved*	—	
0xFF	Other	var	Non-standard.

Operational model: symmetric key vs. KEM

The symmetric cipher always encrypts the data. `KeyEncap` determines how the CEK reaches the intended recipient:

- `KeyEncap`=`0x00` **(passphrase/raw key):** The responder provides a passphrase or externally managed key. Suitable for single-operator scenarios.
- `KeyEncap`≠`0x00` **(KEM):** The acquirer encrypts to the organization's public key. The responder need not know any secret. The private key is held by the analysis team or

in an HSM. This provides separation of duties: the responder who captures evidence cannot decrypt it.

- **Passphrase + KEM (dual control):** Both the responder's passphrase and the organization's private key are required, preventing either party from accessing evidence alone.

The post-quantum options protect against "harvest now, decrypt later" attacks.

Hybrid X25519 + ML-KEM-768. The recommended hybrid scheme (`0x04`) concatenates shared secrets from X25519 and ML-KEM-768 before KDF, ensuring the file remains secure as long as *either* assumption holds. This concatenate-then-KDF construction follows the approach analyzed by Bindel et al. [8] and adopted by TLS 1.3 hybrid key exchange [9]. The hybrid mode is the **recommended** setting for Investigation mode captures.

10.5 Key Derivation

`0x00` — **None**
256-bit CEK provided externally. KDF fields are zero.

`0x01` — **Argon2id**
Passphrase-derived. Minimum: time=3, memory=65536 KiB, lanes=4.

Because `KDFSalt` is generated uniquely per file (16 random bytes), the same passphrase produces a different CEK for each capture, eliminating nonce-reuse risk even when nonces are generated independently across files.

10.6 Interaction with the Integrity Chain

When `Encrypted` is set:

- `PrevHash`: **MUST** be zero in every block, in the capture segment and in extension segments. AEAD authenticates atomically.
- EoC `FileHash` and End-of-Extension `SegmentHash`: **SHOULD** be computed over plaintext (enables post-decryption verification).
- AAD: the file header with bytes `0x78`–`0x7F` (`CipherLen`) set to zero, followed by any KEM ciphertext.
- Extension segments (Section 4.6) are part of the encrypted stream; the chain-continuity check is skipped (all `PrevHash` zero) but `SegmentHash` and `BlockCount` are verified after decryption. Segments added after the primary tag are sealed extension segments (Section 10.8).

10.7 Streaming Encryption

Producers **SHOULD** encrypt incrementally: (1) write the header with `CipherLen`=0 (or its final value, if already known), init AEAD with key, nonce, AAD; (2) for each block, serialize, encrypt, write; (3) after EoC and any extension segments, finalize AEAD, append 16-byte tag; (4) if `CipherLen`=0 was written and the output is seekable, patch the 8 bytes at header offset `0x78` once with the primary ciphertext length. A parallel hasher (`HashAlgo`) computes plaintext `FileHash` and `SegmentHash` before encryption. $O(1)$ memory.

Producers **SHOULD** record `CipherLen`. Producers that know the ciphertext length before encryption begins **MAY** write the final value directly; otherwise they write 0 and, on seekable output, patch it once after the tag. Non-seekable producers that do not know the length leave it 0. The `CipherLen` patch is the only permitted modification of a written header, and only

the producer that seals the primary AEAD stream **MAY** perform it; appenders (Sections 4.6 and 10.8) **MUST NOT**, and the header `BlockCount` is still never patched (producer rule 16). `FileHash` always covers the header with `CipherLen` zero (it is computed before any patch, and a directly written value is hashed as zero); consumers verifying `FileHash` **MUST** hash bytes `0x78`–`0x7F` as zero.

10.8 Sealed Extension Segments

A sealed extension segment adds one extension segment (Section 4.6) to an encrypted container after the primary AEAD tag has been written. Each sealed segment is a separate AEAD message under a per-segment subkey of the CEK, with its own nonce and tag, chained to the preceding tag. Sealed segments require a non-zero header `CipherLen` (Section 10.2).

```
... | Primary AEAD tag | SealHeader 1 | Ciphertext 1 | Tag 1 | ... |
   SealHeader m | Ciphertext m | Tag m | [trailing data]
```

Each `SealHeader` is 64 bytes of cleartext (Table 40), followed by `SegCipherLen` bytes of ciphertext and a 16-byte tag.

Table 40: Sealed segment header (64 bytes).

Off	Size	Field	Abbr	Description
+0x00	4	Magic	Mg	0x4D534C53 ("MSLS").
+0x04	2	SealHdrSize	SHS	64.
+0x06	2	SegIndex	SI	1-based position of this sealed segment after the primary tag.
+0x08	8	SegCipherLen	SCL	Ciphertext length in bytes. **MUST** be > 0.
+0x10	24	Nonce	—	AEAD nonce, encoded as the header `Nonce` for `EncAlgo` (Table 5).
+0x28	24	Reserved	R	Zero.

Size: 4+2+2+8+24+24 = 64.
Cipher suite and key. The cipher suite is the header `EncAlgo`. Segment i is encrypted under $SegKey_i$ = HKDF-BLAKE3(IKM=CEK, salt=`DumpUUID`, info="MSL-SEG-v1" ‖ LE16(i)), the construction of Section 10.4, where the CEK is the Argon2id output, the external key, or the KEM-derived CEK. `SegIndex` therefore ranges from 1 to 65535. `Nonce` **MUST** be generated fresh from a CSPRNG for every segment; the per-segment subkey separates segment keys from the primary stream, and the fresh nonce prevents (key, nonce) reuse if a segment index is written again after a truncation.
Associated data. AAD_i is the file header with bytes `0x78`–`0x7F` (`CipherLen`) set to zero, followed by any KEM ciphertext, followed by the 64-byte `SealHeader` of segment i and the tag of segment $i-1$, where Tag_0 is the primary tag. This binds each segment to its position, to the exact bytes it extends, and to this file, so segments cannot be grafted onto a re-sealed copy with the same `DumpUUID` and passphrase (against parties without the CEK).
Plaintext. The plaintext is exactly one extension segment per Section 4.6: `PrevHash` **MUST** be zero, the End-of-Extension block **MUST** end exactly at `SegCipherLen`, the permitted-block-type rules apply, and `SegmentHash` and `BlockCount` are verified after decryption.
Producer rules. A producer appending a sealed segment **MUST** hold the CEK (in KEM mode this requires the recipient's private key, so an acquirer that only holds the public key cannot append); **MUST** verify the primary tag and every existing sealed segment; **MUST** refuse when the header `CipherLen`=0; **SHOULD** refuse when trailing data is present; **MUST NOT** modify any earlier byte, including the header; and **MUST** write the whole sealed segment (`SealHeader`, ciphertext, tag) as one unit.

Consumer rules. After the primary tag verifies, a consumer parses sealed segments in order, $i = 1, 2, \ldots$. On any failure—short read, `Magic` or `SealHdrSize` mismatch, `SegIndex`$\neq i$, `SegCipherLen` zero or exceeding the bytes remaining after the `SealHeader` minus 16, tag failure, or a malformed plaintext segment—everything from that segment's first byte to EOF is trailing data. A consumer **MUST NOT** present plaintext from a sealed segment before its tag verifies.
Security considerations. Dropping trailing sealed segments is undetectable, exactly as for plaintext extension segments (Section 4.6); this is acceptable because evidence block types are forbidden in segments. Sealed segments never alter the capture segment or the primary ciphertext.

11 Raw Dump Import

An MSL file may be created by importing a pre-existing raw memory dump (ELF core, Windows Minidump, ProcDump output, or plain byte stream) rather than by live acquisition. The Import Provenance block (`0x0030`) records the origin of the imported data: the source format, the tool that performed the conversion, the original file size, and an optional free-text note. When `Imported` (bit 0) is set in `Flags`, this block **MUST** appear as Block 0.

Table 41: Import Provenance payload.

Off	Size	Field	Description
+0x00	2	SourceFormat	Table 42.
+0x02	2	Reserved	Zero.
+0x04	4	ToolNameLen	Incl. null.
+0x08	8	ImportTime	ns since epoch.
+0x10	8	OrigFileSize	0 if unknown.
+0x18	4	NoteLen	0 if none.
+0x1C	4	Reserved2	Zero.
+0x20	var	ToolName	UTF-8, pad8.
$+t$	var	Note	UTF-8, pad8.

Table 42: Source format codes.

Code	Format
0x0000	Unknown
0x0001	Raw byte stream
0x0002	ELF core dump
0x0003	Windows Minidump
0x0004	macOS core dump
0x0005	ProcDump
0xFFFF	Other

Procedure: (1) Set `Flags` bit 0. (2) Block 0: Import Provenance. (3) Regions from source. (4) Modules if available; Thread Contexts from source thread records if available (Section 5.7). (5) VAS Map with `NameSource`=`0x02` when the source carries mapping metadata (Section 5.8). (6) Hash chain and EoC.

12 Parsing Walkthrough

Step 1:
Read 9 bytes. Verify magic. `Endianness` **MUST** be `0x01`. Read `HeaderSize`.

Step 2:
If `Encrypted`: read extension (`0x40`–`0x7F`). If `KEMCtLen`>0, read KEM ciphertext. Obtain key. Decrypt. Continue with plaintext.

Step 3:
Read blocks at `HeaderSize` (+`KEMCtLen` if encrypted) up to the EoC block. Verify `BlockLength` $\geq$ 80. Parse or skip via `BlockLength`. Verify every non-zero `BlockCount` (header, EoC) against the parsed capture-segment count.

Step 4:
If unencrypted: verify the `PrevHash` chain with the algorithm selected by `HashAlgo`. If encrypted: locate the AEAD tag at `HeaderSize`+`KEMCtLen`+`CipherLen` (or at EOF−16 when `CipherLen`=0), rejecting an out-of-bounds `CipherLen`, and verify it.

Step 5:
Verify EoC `FileHash` if present. Then parse extension segments (Section 4.6): verify chain continuity, `SegmentHash`, and `BlockCount` of each. If encrypted and `CipherLen`≠0, then parse sealed extension segments after the tag (Section 10.8). The remainder is trailing data.

Step 6:
`Investigation`? Expect Block 2 = System Context.

Step 7:
`Imported`? Read Import Provenance.

13 Worked Example

Module Entry block for `ntdll.dll` `v10.0.22621.1` at `0x7FFDD4D00000`. Path=30B (pad8=32). Version=13B (pad8=16). Payload=496B. Block=576 (`0x0240`).

Table 43: ntdll.dll Module Entry.

Off	Field	Hex	Value
— Header (80B) —			
0x00	Magic	4D 53 4C 43	MSLC
0x04	BlockType	02 00	Module Entry
0x08	BlockLength	40 02 00 00	576
0x0C	PayloadVersion	01 00	1
0x10	BlockUUID	(16B)	UUIDv4
0x20	ParentUUID	(16B)	Module List UUID
0x30	PrevHash	(32B)	BLAKE3 of prev
— Payload (496B) —			
0x50	BaseAddr	00 00 D0 D4 FD 7F 00 00	0x7FFDD4D00000
0x58	ModuleSize	00 00 20 00 00 00 00 00	2 MiB
0x60	PathLen	1E 00	30
0x62	VersionLen	0D 00	13
0x68	Path (32B)	43 3A 5C 57...	ntdll.dll
0x88	Version (16B)	31 30 2E 30...	10.0.22621.1
0x98	DiskHash (32B)	...	BLAKE3 of binary
0xB8	BlobLen	80 01 00 00	384
0xC0	NativeBlob	(384B)	LDR_DATA_TABLE_ENTRY

14 Conformance

14.1 Producer

A conformant producer **MUST**: (1) write valid header with `Magic`, `Endianness`=`0x01`, `HeaderSize`, `Version`; (2) generate UUIDv4 in RFC 9562 binary layout; (3) compute `PrevHash` with the selected `HashAlgo` (zero when encrypted); (4) set reserved fields to zero; (5) pad8 all variable fields; (6) never modify written blocks; (7) set `CapBitmap` accurately; (8) if importing: set `Flags` bit 0, emit Import Provenance as Block 0; (9) set `PayloadVersion` to 1; (10) set `CompAlgo` bits to 00 when uncompressed; (11) split regions exceeding $2^{32}-81$ bytes; (12) write 32 zero bytes for unavailable `DiskHash`; (13) reject `PageSizeLog2` outside $[10, 40]$; (14) emit well-formed UTF-8 per RFC 3629 for all string fields; (15) convert port numbers from network byte order to little-endian before writing; (16) set the header `BlockCount` to the number of capture-segment blocks including EoC, or to zero when unknown at header-write time, and never patch it afterwards; record the exact count in the EoC `BlockCount` field.

A conformant acquirer **MUST** emit Process Identity as Block 0 and Module List Index as Block 1.

Investigation: (17) emit System Context as Block 2; (18) set `CapBitmap` bit 11; (19) set `TableBitmap`; (20) set `Redacted` flag (bit 3) when any field has been intentionally stripped or pseudonymized.

Encryption: (21) `HeaderSize`=128; (22) valid encryption extension; (23) AEAD with header (+KEM ct) as AAD, bytes `0x78`–`0x7F` zeroed (Section 10.2); (24) append 16-byte tag immediately after the ciphertext, and append after it only sealed extension segments (Section 10.8); (25) zero all `PrevHash`; (26) `Nonce` from CSPRNG; (27) KEM ciphertext after header when `KeyEncap`≠`0x00`.

Compression: (28) when `Compressed` is set, write 8-byte `UncompressedSize` before compressed data; (29) set `BlockLength` to on-disk size.

Block types: (30) **MUST NOT** emit block types whose payload format is not yet specified in this document, except producer-defined Structural types inside extension segments (Section 4.6).

Hash algorithm: (31) set `HashAlgo` to a registered code from Table 12; the default (`0x00`, BLAKE3) **SHOULD** be used unless FIPS compliance, organizational policy, or platform performance measurements indicate otherwise; (32) use the selected algorithm exclusively for all `PrevHash`, `FileHash`, `SegmentHash`, and `DiskHash` computations within the file; **MUST NOT** mix algorithms within a single file.

Extension segments: (33) when appending, **MUST NOT** modify any byte of the header, the capture segment, or an existing extension segment; (34) terminate every extension segment with an End-of-Extension block carrying the exact `SegmentHash` and `BlockCount`; (35) continue the `PrevHash` chain from the last block already in the file (zero when encrypted); (36) emit only the block types Section 4.6 permits in extension segments; (37) in encrypted containers, write extension segments inside the AEAD stream before the tag; after the tag, append only sealed extension segments (Section 10.8).

Thread Context: a producer that captures thread register state **SHOULD** emit one Thread Context block (`0x0011`) per thread, set the `Current` flag on exactly one of them, set `CapBitmap` bit 2, and include at least the program-counter and stack-pointer registers (Section 5.7). (38) Thread Context blocks **MUST** be written in the capture segment and **SHOULD** precede the first Memory Region; (39) set `Suspended` only for registers read from a stopped thread and `Partial` whenever the register file is incomplete.

Ciphertext length: (40) write `CipherLen` either as its final value or as 0; when 0 is written and the output is seekable, the sealing producer **MAY** patch it once after the tag (Section 10.7); no other party patches it.

Sealed extension segments: (41) when appending after the primary tag, write only sealed extension segments per Section 10.8: hold the CEK, verify the primary tag and every existing sealed segment, refuse when the header `CipherLen`=0, use $SegKey_i$ with a fresh CSPRNG `Nonce`, write each segment as one unit, and never modify earlier bytes.
Register names: (42) name Thread Context registers per Table 24 for the architectures it lists, and never write aliases (ARM64 `fp`/`lr`; ARM32 `r13`–`r15`).
A producer **SHOULD** emit an EoC block. `FileHash` **SHOULD** be computed over plaintext even when encrypted. When encrypted, the producer **SHOULD** record an external file hash for chain-of-custody.

14.2 Consumer

A conformant consumer **MUST**: (1) reject invalid magic or `Endianness` $\neq$ `0x01`; (2) use `HeaderSize` to locate blocks; (3) skip unknown types via `BlockLength`; (4) reject `BlockLength` < 80; (5) detect end-of-blocks; (6) ignore reserved fields; (7) reject type `0x0000`; (8) skip unrecognized `PayloadVersion` for known block types with a logged warning indicating which block types were affected; (9) treat all-zero `DiskHash` as unavailable; (10) reject `PageSizeLog2` outside $[10, 40]$; (11) use explicit length fields to determine string extent, not NUL-scanning beyond the declared length; (12) preserve raw bytes for malformed UTF-8 fields and not reject the entire block; (13) verify that every non-zero block count—header `BlockCount` and EoC `BlockCount`—equals the number of capture-segment blocks parsed including the EoC block, and otherwise report the file as truncated or corrupt; accept a 40-byte EoC payload as count unknown; (14) after a verified EoC block (and, when encrypted, the primary AEAD tag), attempt to parse extension segments per Section 4.6 and, when `CipherLen`$\neq$0, sealed extension segments per Section 10.8, and treat everything after the last verified segment as trailing non-MSL data.
Encryption: (15) verify `HeaderSize`=128; read KEM ct; decrypt via AEAD with AAD (with `CipherLen` zeroed, Section 10.2); reject on failure; (16) skip `PrevHash` verification.
Investigation: (17) expect Block 2 = System Context; check `TableBitmap`; (18) when `Redacted` is set, interpret zero-length optional strings as intentionally redacted rather than absent.
Compression: (19) read 8-byte `UncompressedSize` before inflating; (20) reject if decompressed size does not match `UncompressedSize`.
Cross-references: (21) log and continue on dangling `ParentUUID` or payload-embedded UUID references; treat the affected block as an orphan but **MUST NOT** discard it.
Hash algorithm: (22) **MUST** implement at minimum `HashAlgo`=`0x00` (BLAKE3) and `HashAlgo`=`0x01` (SHA-256); **MAY** implement `0x02` (SHA-512/256) and other registered codes; **MUST** reject files whose `HashAlgo` the consumer does not implement, emitting a diagnostic indicating the unsupported algorithm code.
Extension segments: (23) verify chain continuity from the EoC block (unencrypted files), `SegmentHash`, and `BlockCount` of every extension segment, and **MUST NOT** present an unterminated or failing segment as verified; (24) accept Structural and producer-defined block types in verified segments, skipping unrecognized payloads via `BlockLength`; (25) honour the last VAS Map in file order (Section 5.8) and merge Module Build-ID Manifest rows by `BaseAddr` (Section 5.10).
Page states: (26) decode `PageStateMap` value `11` as NotResident (a zero-content page, Section 7), never as a read failure.
Ciphertext length: (27) locate the AEAD tag at EOF-16 when `CipherLen`=0, otherwise at `HeaderSize`+`KEMCtLen`+`CipherLen`; reject `CipherLen` $>$ file size $-$ `HeaderSize` $-$ `KEMCtLen` $-$ 16 (Section 10.2).
Sealed extension segments: (28) verify sealed segments in order per Section 10.8; **MUST NOT** present plaintext from a sealed segment before its tag verifies; treat everything from the

first failing segment to EOF as trailing data.
Register names: (29) **SHOULD** normalise ARM64 `fp`→`x29` and `lr`→`x30` in Thread Context blocks written by 1.1.0 producers.
A consumer **SHOULD**: verify the `PrevHash` chain when unencrypted; verify EoC `FileHash` after decryption.

15 Revision History

1.1.1 — 2026-10-23. Patch revision; the header `Version` remains `0x0101` (Section 3.4).

- `CipherLen` (header offset `0x78`, formerly `Reserved3`; Sections 3.2, 10.2, 10.7): byte length of the primary AEAD ciphertext. The AAD is now the file header with bytes `0x78`–`0x7F` (`CipherLen`) set to zero, followed by any KEM ciphertext; for `CipherLen`=0 it is byte-identical to the 1.1.0 AAD. Producers write the final value or 0; a seekable sealing producer that wrote 0 patches it once after the tag, the only permitted header patch. A 1.1.0 consumer cannot decrypt files with `CipherLen`≠0; files with `CipherLen`=0 are unaffected. Producer rules 23, 24, 40; consumer rule 27.
- **Sealed extension segments** (Section 10.8, Table 40): an encrypted container can be enriched after sealing by appending AEAD segments under per-segment HKDF-BLAKE3 subkeys of the CEK, each with a 64-byte `SealHeader` (ciphertext length `SegCipherLen`, nonce) and its own tag, chained to the preceding tag; requires a non-zero header `CipherLen`. Replaces the 1.1.0 rule that nothing may be appended after the tag. Producer rules 37, 41; consumer rules 14, 28.
- **Canonical register mnemonics** (Section 5.7, Table 24): register names for x86_64, x86, ARM64 and ARM32 are normative; aliases (`fp`, `lr`, `r13`–`r15`) are forbidden; consumers normalise `fp`→`x29`, `lr`→`x30`. Producer rule 42; consumer rule 29.
- **Erratum:** 1.1.0 permitted trailing data after the AEAD tag but gave no means to locate the tag; the tag is now located via `CipherLen` (Sections 10.2, 12).

1.1.0 — 2026-10-23. Minor revision; 1.0.0 files remain valid, consumers **MUST** accept any minor of major 1 (Section 3.4).

- **End-of-Capture** `BlockCount` (Section 4.5): the EoC payload now carries the exact capture-segment block count at `+0x28`; the header `BlockCount` is never patched after writing (Section 3.1). A 40-byte legacy EoC payload remains accepted.
- **Extension segments** (Section 4.6): blocks appended after the EoC block are grouped into segments terminated by a new End-of-Extension block (`0x0FFE`) with `SegmentHash`, `WrittenAt`, `BlockCount`, and `Producer`; the `PrevHash` chain continues across the EoC block. Evidence block types are forbidden in segments; producer-defined Structural payloads (e.g., Pointer Graph `0x1003`) are permitted only there. Bytes after the last verified segment are trailing data.
- **VAS Map** (`0x1001`, Section 5.8): payload defined (`Count`/`EntrySize` preamble, 24-byte entry head, `NameSource` at entry offset `+0x14`); one entry per enumerated range; the last map in file order supersedes.
- **Page state NotResident** (`11`, Section 7): the former reserved value now denotes a mapped, never-touched page with zero content; Section 7 renamed to "Page State Map"; design principle 2 reads "four-state".

- **New capture-time blocks**: Physical Memory Map (`0x0059`, Section 5.9, not yet emitted by any known producer), Module Build-ID Manifest (`0x005A`, Section 5.10), Region Omission Manifest (`0x005B`, Section 5.11); Connectivity Table (`0x0054`, Section 6.6) with `TableBitmap` bit 3 (Table 30).
- `HashAlgo` (header offset `0x3D`, Section 4.4, Table 12): integrity hash selector (BLAKE3 default, SHA-256, SHA-512/256); producer rules 31–32 and consumer rule 22. Previously present only in the LaTeX edition.
- **Thread Context** (`0x0011`, Section 5.7, Tables 22/23): per-thread register files contributed by Ricardo J. Rodríguez (PR #1), with `Arch` at `+0x13`, `Suspended`/`Partial` flags, thread states `0x05`–`0x07`, and register flags for thread pointer and link register.
- Markdown and LaTeX editions synchronized: paper title, `HashAlgo`, Connectivity Table renumbered from `0x0055` to `0x0054`.

1.0.0 — 2026-07. Initial working draft.

References

[1] S. Bradner, “Key words for use in RFCs to Indicate Requirement Levels,” RFC 2119, 1997.

[2] K. Davis, B. Peabody, P. Leach, “Universally Unique IDentifiers (UUIDs),” RFC 9562, 2024.

[3] F. Yergeau, “UTF-8, a transformation format of ISO 10646,” RFC 3629, 2003.

[4] J. O’Connor et al., “BLAKE3: One function, fast everywhere,” 2020. https://github.com/BLAKE3-team/BLAKE3-specs/blob/master/blake3.pdf

[5] NIST, “Secure Hash Standard (SHS),” FIPS 180-4, 2015. https://doi.org/10.6028/NIST.FIPS.180-4

[6] TIS Committee, “ELF Specification,” v1.2, 1995.

[7] NIST, “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” FIPS 203, 2024. https://doi.org/10.6028/NIST.FIPS.203

[8] N. Bindel, J. Brendel, M. Fischlin, B. Goncalves, D. Stebila, “Hybrid Key Encapsulation Mechanisms and Authenticated Key Exchange,” in *Post-Quantum Cryptography (PQCrypto)*, Springer, 2019.

[9] D. Stebila, S. Fluhrer, S. Gueron, “Hybrid key exchange in TLS 1.3,” Internet-Draft draft-ietf-tls-hybrid-design, IETF, 2024.